You said:

hey hloo

ChatGPT said:

Heyyy! 👋😄
What's up? What are we working on today?

You said:

I want to build AI projects

ChatGPT said:

Yesss 😎🔥 If your goal is to build **AI projects that actually look strong on your resume**, we should avoid the usual *"chatbot + OpenAI API"* projects.

Since you already have development experience, I'd suggest building projects where **AI is the core of the system**, not just an API pasted into a web app.

🚀 Projects I'd consider for you

Project	AI/ML Core	Difficulty	Resume Value
🧠 **AI Interviewer + Skill Evaluator**	NLP + speech + scoring	⭐⭐⭐⭐	🔥🔥🔥
👁️ **Real-Time Driver Drowsiness & Distraction Detection**	Computer Vision	⭐⭐⭐⭐	🔥🔥🔥
📰 **Fake News / Deepfake Detection System**	NLP + Vision	⭐⭐⭐⭐⭐	🔥🔥🔥
🏥 **Medical Report Understanding System**	NLP + RAG	⭐⭐⭐⭐	🔥🔥🔥
🏭 **Predictive Maintenance from Sensor Data**	Time-series ML	⭐⭐⭐⭐	🔥🔥🔥
🤖 **AI Agent for Software Debugging**	LLM + code analysis	⭐⭐⭐⭐⭐	🔥🔥🔥
🗺️ **AI Traffic Incident Detection**	Computer Vision	⭐⭐⭐⭐	🔥🔥🔥
🦯 **Vision-Based Assistive AI**	CV + speech	⭐⭐⭐⭐⭐	🔥🔥🔥

Project	AI/ML Core	Difficulty	Resume Value
📄 **Intelligent Resume Screening + Job Matching**	NLP + embeddings	⭐⭐⭐⭐	🔥🔥
🧠 **Personalized Learning Prediction Engine**	ML + recommendation	⭐⭐⭐⭐	🔥🔥

But I want to make this more interesting for you.

Instead of immediately choosing a project, we can build **one serious AI project from zero**, including:

Problem → dataset → ML model → training → evaluation → backend → frontend → deployment → GitHub → resume bullet points

And I'll teach you what every part does rather than giving you a black-box implementation.

For example, we could build:

> **Real-Time Driver Safety AI**
> Webcam → detect face/eyes → estimate drowsiness → detect phone usage → detect distraction → risk score → warning system → dashboard → historical analytics.

That gives you **Computer Vision + Deep Learning + real-time inference + backend + deployment**, which is much stronger than a simple AI chatbot.

Or we can go in a completely different direction with **LLMs/RAG/AI agents**, **computer vision**, **ML/data science**, or **AI + systems**.

If you're building specifically for **placements/internships**, I'd lean toward **2–3 substantial AI projects rather than 8 small ones**.

You said:

great how are these projects different from those Here's the problem with most "AI project ideas" you see online: they're basically the same project in different forms. Eg: A support chatbot with RAG

Recruiters and hiring managers have already seen hundreds of versions of this project.

It doesn't necessarily show that you understand the harder parts of building AI systems: what happens when retrieval fails, an agent gets

stuck in a loop, the model produces unsafe output, latency becomes a problem, or the system has to work reliably for real users.

So instead of giving you another recycled list, I want to focus on 5 projects:

Which can actually help you build DEEPER skills,

Which are designed around TRENDING skills companies are actively hiring for

Which can help you solve the same problems AI engineers are being hired to solve in PRDOUCTION.

If you like my work and find my resources useful, consider subscribing to this Newsletter to get them in your mailbox weekly :)

bhavyasrigali.nitandhra@gmail.com
password
bhavyasrigali.nitandhra@gmail.com
Subscribe
1. RAG Project
GitContext: End-to-End Codebase Intelligence & Developer Copilot
AI Skills: RAG, embeddings, vector databases, hybrid retrieval, reranking

Use case: Help engineers understand unfamiliar or legacy codebases without manually digging through code, PRs, commits, and documentation.

Build:

Ingest an entire GitHub repository: source code, README/docs, commit history, PR discussions, and incident reports.

Parse and chunk code intelligently by function, class, and module, rather than arbitrary token counts.

Generate embeddings and store them in Pinecone/Qdrant/Weaviate.

Implement hybrid retrieval: vector search + BM25/keyword search for exact error messages and function names.

Add reranking before sending retrieved context to the LLM.

Attach metadata such as file path, commit, author, PR, and last-modified date.

Detect when indexed code becomes stale and automatically re-index changed files.

Build a FastAPI backend + web/CLI interface where developers can ask questions about the repository.

Evaluate: Recall@K, retrieval precision, answer correctness, citation accuracy, latency.

Resume angle: Built an end-to-end RAG developer copilot over source code, Git history and PR discussions using hybrid retrieval and reranking, achieving X% Recall@5 with Y ms retrieval latency.

2. AI Agent Project
ResolveAI: End-to-End Autonomous Customer Resolution Agent
AI Skills: Agentic AI, tool calling, workflow orchestration, human-in-the-loop

Use case: Automatically investigate and resolve customer refund requests while escalating risky cases to humans.

Build:

Give the agent tools for order lookup, refund-policy search, customer history, fraud checks, and refund creation.

Implement a state-machine/graph workflow using LangGraph or a similar orchestration framework.

Let the agent reason across multiple steps: identify problem → gather evidence → check policy → recommend resolution → execute/escalate.

Add short-term state so the agent remembers information collected earlier in the workflow.

Define clear auto-resolution vs. human-escalation rules.

Add retry limits and loop detection so an agent cannot continue indefinitely.

Log every tool call, decision, escalation, and final outcome.

Build an operator dashboard where humans can approve/reject escalated actions.

Evaluate: autonomous resolution rate, escalation accuracy, tool-call success rate, average steps per case, latency and cost per resolution.

Resume angle: Designed an agentic customer-resolution system with multi-tool orchestration and human-in-the-loop controls, autonomously resolving X% of simulated cases while escalating high-risk actions.

3. LLM Project
ActionGraph: End-to-End Meeting Intelligence Platform
AI Skills: LLM application engineering, structured outputs, prompt engineering, evaluation

Use case: Convert messy meeting transcripts into reliable decisions, action items, owners, deadlines, blockers, and follow-ups.

Build:

Accept uploaded transcripts or generate them from meeting audio using speech-to-text.

Define a structured schema: action | owner | deadline | confidence | evidence.

Extract both explicit commitments (“I’ll finish this Friday”) and implicit commitments (“I’ll take a look”).

Use structured LLM outputs rather than parsing free-form responses.

Force every extracted action item to include the exact transcript evidence supporting it.

Add confidence scoring and route ambiguous items to human review.

Store historical meetings so unresolved actions can carry into future meetings.

Build a dashboard showing meetings, decisions, action items, owners, and overdue tasks.

Evaluate: precision, recall and F1 separately for explicit vs. implicit action items, plus hallucination rate.

Resume angle: Built an end-to-end LLM meeting intelligence platform extracting structured decisions and action items with X% F1, evidence-based citations and confidence-driven human review.

4. AI Guardrails Project
End-to-End Prompt-Injection Detection & Guardrail Gateway
AI Skills: AI security, prompt injection, guardrails, authorization, adversarial evaluation

Use case: Protect AI agents that consume untrusted emails, webpages and documents before they can access sensitive data or execute external actions.

Build:

First create a deliberately vulnerable agent with access to documents and tools.

Build an adversarial test set containing direct and indirect prompt-injection attacks.

Treat retrieved documents as untrusted data rather than system instructions.

Add an input/output security layer that detects suspicious instructions and sensitive-data leakage.

Introduce tool permissions so reading information and executing actions have different privilege levels.

Require human approval for high-risk actions such as sending data

externally.

Log blocked attacks, allowed actions, tool calls and security decisions.

Build a red-team evaluation suite that automatically attacks the system after every change.

Evaluate: attack success rate, false-positive rate, false-negative rate and legitimate-task completion rate before vs. after guardrails.

Resume angle: Built an AI security gateway protecting tool-using agents against direct and indirect prompt injection, reducing adversarial attack success from X% to Y% across Z red-team scenarios.

5. Fine-Tuning Project
ModelRoute: End-to-End LLM Distillation, Fine-Tuning & Inference Platform
AI Skills: Fine-tuning, LoRA/QLoRA, model optimization, inference and model routing

Use case: Replace expensive frontier-model calls for repetitive tasks with a smaller specialized model while maintaining acceptable quality.

Build:

Choose a measurable task such as support-ticket intent classification and routing.

Create a labelled dataset containing thousands of realistic support tickets.

Establish a baseline using a larger frontier model.

Fine-tune a smaller open-weight model using LoRA/QLoRA.

Build an evaluation dataset that is completely separated from training data.

Compare the models on accuracy/F1, latency, token usage and cost.

Serve the fine-tuned model behind a production API.

Add a router: use the smaller model for high-confidence requests and fall back to the larger model for difficult cases.

Evaluate: F1/accuracy, p95 latency, cost per 1,000 requests, fallback rate and quality degradation versus the baseline.

Resume angle: Fine-tuned and deployed a specialized open-weight LLM using QLoRA and confidence-based model routing, reducing inference cost by X% and p95 latency by Y% while maintaining Z% of baseline quality.

Final Thoughts
Build one project deeply, measure how well it works, understand where it fails, and you'll have a much stronger interview story than someone with five half-finished AI demos.

ChatGPT said:

Ahhh, **yes — this is a much better benchmark** for what you mean by "AI projects." And you're right to ask the distinction.

The projects I initially suggested are mostly **AI application projects**. The 5 you pasted are much closer to **AI engineering / production AI systems**.

The real difference

Think of it like this:

Basic AI project:

> "I used a model to solve X."

AI engineering project:

> "I built a system around a model, measured it, deliberately broke it, handled failures, optimized it, secured it, and deployed it."

That second category is what makes those 5 projects interesting for **AI Engineer / ML Engineer / Backend + AI roles**.

Let's compare them directly

	Typical project I suggested	Your 5 projects
Main focus	Applying AI to a problem	**Building reliable AI systems**
Model	Usually central	One component of a larger system
Evaluation	Often basic accuracy/demo	**Formal metrics + failure analysis**
Failure handling	Limited	**Core requirement**
Production concerns	Usually minimal	**Latency, cost, security, reliability**
Backend	Supporting component	**Part of the AI architecture**
AI infrastructure	Limited	**Retrieval, routing, orchestration, inference**
Security	Usually absent	**Explicitly addressed**
Human-in-loop	Rare	Important
Resume story	"Built an AI application"	**"Engineered an AI system achieving X under Y constraints"**

And the 5 you pasted are actually five different AI engineering directions

This is the part I really like.

1. GitContext → RAG / Retrieval Engineering

It's not:

> "I made a chatbot that answers questions about PDFs."

It's:

```
GitHub Repository
       ↓
Code Parser
       ↓
Function/Class/Module chunks
       ↓
Embeddings ──────┐
                 ├── Hybrid Retrieval
BM25 ────────────┘
       ↓
   Reranker
       ↓
Relevant code + metadata
       ↓
      LLM
       ↓
Answer + citations
```

And then you ask:

Does retrieval actually work?

That's where things become serious.

You measure:

- Recall@K
- Precision
- citation accuracy
- answer correctness
- retrieval latency
- stale-index detection

That's **retrieval engineering**, not merely "using RAG."

---

2. ResolveAI → Agent Engineering

A normal AI agent project:

> "Give an LLM tools and let it do things."

A serious agent project asks:

What happens when it makes a wrong decision?

For example:

```
Customer: "I want a refund."

        ↓

Agent
   ↓
Order lookup
   ↓
Policy lookup
   ↓
Fraud check
   ↓
Customer history
   ↓
Decision
   ↓
  ┌─────────────────┐
  │                 │
Safe            Risky
  │                 │
  ↓                 ↓
Refund        Human approval
```

Now you have to solve:

- state management
- tool failures
- retries
- infinite loops
- authorization
- escalation
- observability
- human approval

That's much closer to what companies actually worry about when deploying agents.

---

3. ActionGraph → LLM Application Engineering

This one is subtle.

It's not really about building a fancy chatbot.

The important question is:

> **Can I force an LLM to produce reliable structured information from messy human language?**

For example:

> "I'll send the final API documentation to Ravi by Friday, but someone needs to review the authentication section first."

Expected:

```
{
  "action": "Send final API documentation",
  "owner": "Speaker",
  "deadline": "Friday",
  "blocker": "Authentication section needs review",
  "confidence": 0.91,
  "evidence": "..."
}
```

Then you evaluate whether that extraction is actually correct.

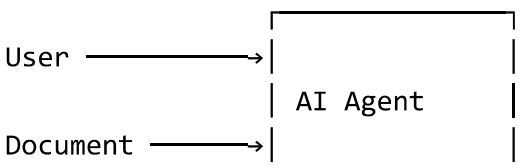
That's **LLM reliability** + **structured output** + **evaluation**, rather than just prompt engineering.

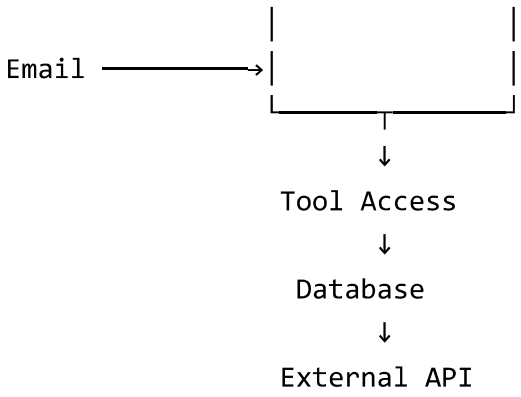
---

4. Guardrail Gateway → AI Security

This is a completely different level.

You intentionally build:





Then you attack your own system.

For example:

> A malicious instruction hidden inside a document tries to convince the agent to expose confidential information.

Now your project becomes:

Build → Attack → Detect → Block → Measure → Improve

That's **AI security / adversarial engineering**.

This is particularly interesting because most student projects don't even think about this layer.

---

5. ModelRoute → Model Optimization / Inference Engineering

This one is probably the most different from typical college AI projects.

Instead of:

> "I fine-tuned an LLM."

You're asking:

> "Can I make an AI system cheaper and faster without significantly hurting quality?"

For example:

Before

```
Every request
      ↓
Large expensive model
      ↓
$X / request
```

After

```
                Request
                   ↓
              Small Model
                   ↓
              Confidence?
               /       \
            High        Low
             ↓           ↓
         Answer     Large Model
```

Now you measure:

- F1
- quality degradation
- p95 latency
- inference cost
- fallback rate

That is **ML systems / inference engineering**.

So if I were choosing projects for YOU...

I would actually **change my original recommendation**.

Given that you're interested in software development, backend/infrastructure, AI/ML, and placements, I wouldn't tell you to build five unrelated AI applications.

I'd build **3 deep projects**, each demonstrating a different layer.

🥇 Project 1 — GitContext

AI + Backend + RAG + Retrieval

You learn:

```
Embeddings
Vector DB
BM25
Hybrid search
Reranking
Code parsing
Metadata
Evaluation
FastAPI
Caching
Indexing
```

This gives you a very strong **AI Engineer + Backend** project.

🥈 Project 2 — ResolveAI

AI Agents + Systems

You learn:

```
LLM
Tool calling
State machines
LangGraph
Workflow orchestration
Retries
Human-in-the-loop
Observability
Authorization
```

This demonstrates that you understand **agents beyond "I used LangChain."**

🥉 Project 3 — ModelRoute

Fine-tuning + ML Systems

You learn:

```
Dataset creation
Baseline models
```

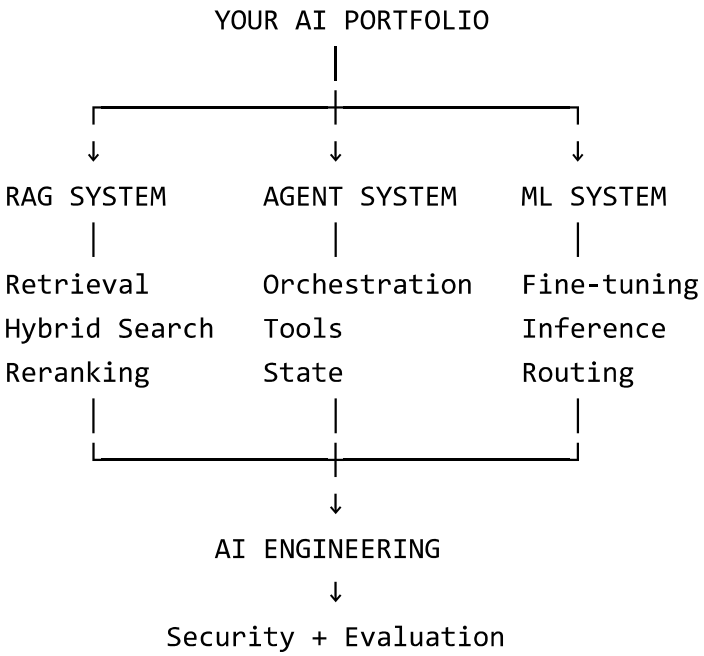
```
LoRA
QLoRA
Fine-tuning
Evaluation
Quantization
Inference
Model serving
Routing
Latency
Cost optimization
```

This gives you the deeper **ML engineering** side.

And there's a really interesting fourth option

I'd personally consider **Guardrail Gateway** as a replacement for one of the above if you're interested in security.

Because then your portfolio becomes:



That's a **much stronger portfolio narrative** than:

> chatbot + sentiment analysis + image classifier + recommendation system + PDF summarizer.

One more important thing

The article you pasted is **not magically good just because it uses buzzwords**.

The *idea* is good, but the difference will come from how deeply you implement it.

For example:

❌ **Weak GitContext**

> "I used Pinecone and Gemini to chat with GitHub repositories."

✅ **Strong GitContext**

> "I compared semantic, BM25, hybrid and reranked retrieval on a benchmark of 500 code questions, analyzed Recall@5 and citation accuracy, implemented incremental indexing for changed files, and measured p95 retrieval latency."

The second person has something interesting to discuss in an interview.

That's the standard I would use for your projects.

And honestly, **I think we should build your AI portfolio using this philosophy rather than the generic project-list approach.**

You said:

great now guide with knowledge on how to build the first project - remember I am not doing this for copy-paste experinece but i am doing it for knowledge and to explain my understanding to the interviewer and to build more projects in future and this is my AI related project so guide me so that it gets intresting and very exicted to learn new things

ChatGPT said:

Absolutely. **This changes how I'll teach you.**

We are **not** going to do:

> "Install this → paste this code → run → GitHub → done."

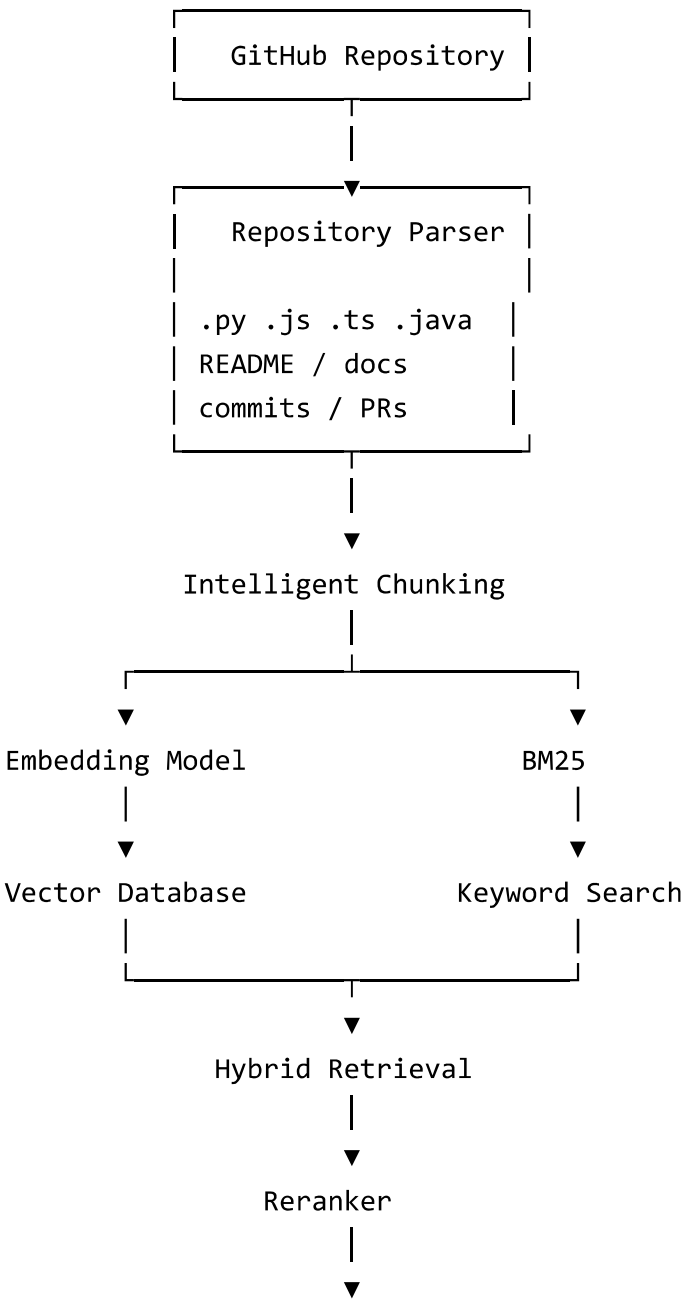
We are going to treat **GitContext as your first serious AI engineering project**, where the project itself becomes a way for you to learn the entire AI stack.

And I'll deliberately make you **struggle productively** at certain points—because if you understand *why* something is needed, you'll be able to build your second, third, and tenth AI project without me.

🧠 Project 1: GitContext

End-to-End Codebase Intelligence & Developer Copilot

Our final system will look roughly like this:



```
Relevant Context
        │
        ▼
      LLM
        │
        ▼
Answer + Code Citations
```

But **we are NOT starting by building this whole diagram.**

First, you need to understand why each box exists.

Phase 0 — What are we actually trying to solve?

Imagine I give you this repository:

```
my-company-project/

├── backend/
│   ├── auth/
│   │   ├── login.py
│   │   └── jwt.py
│   ├── payments/
│   │   ├── payment.py
│   │   └── refund.py
│   └── database.py
│
├── frontend/
│   ├── components/
│   └── pages/
│
├── tests/
├── README.md
└── requirements.txt
```

Now you're a new developer joining the company.

You ask:

> "Where is authentication handled?"

Easy.

Then:

> "What happens when a payment fails?"

Harder.

Then:

> "Why does `process_payment()` call `validate_transaction()` before `create_payment()`?"

Hard.

Then:

> "When was this behavior introduced?"

Now you need Git history.

Then:

> "Was there a bug related to this?"

Now you need commits, PRs, discussions, maybe incident reports.

That's the problem.

GitContext isn't fundamentally a chatbot.

It's a system that tries to answer:

> **"Give me the right pieces of a software system so an LLM can reason about that system."**

That distinction is extremely important.

Phase 1 — Learn the first fundamental concept: RAG

Before writing anything, I want you to understand this.

Suppose you ask an LLM:

> "What does my `payment_service.py` do?"

The model doesn't magically know your private repository.

You could put the entire repository into the prompt.

But that's terrible.

Imagine:

```
Repository = 500,000 tokens

Question = 10 tokens
```

Sending everything is:

- expensive
- slow
- potentially beyond context limits
- noisy

So we need a mechanism to find **only the relevant information**.

That's:

Retrieval-Augmented Generation

RAG.

The basic idea:

```
Question
   ↓
Find relevant information
   ↓
Give information + question to LLM
```

```
    ↓
Answer
```

That's it.

But here's where your learning begins.

🧪 Your first experiment

Don't build GitContext yet.

I want you to experience the problem yourself.

Take a small codebase.

For example:

```
calculator/
├── calculator.py
├── history.py
└── main.py
```

Put several functions inside it.

Then ask yourself:

> "If I give an LLM the entire project, can it answer questions?"

Probably yes.

Now imagine:

```
100 files
10,000 lines
```

Then:

```
10,000 files
1 million lines
```

You immediately encounter the problem.

How do we decide which pieces to give the model?

That question leads to retrieval.

Phase 2 — Embeddings

Now we're going to encounter one of the most important concepts in modern AI engineering.

Suppose we have:

```
"How do I authenticate a user?"
```

and:

```
def verify_jwt(token):
    ...
```

These strings don't share many words.

A simple keyword search may fail.

But semantically:

> authenticate user

and

> verify JWT

are related.

We want computers to represent **meaning numerically**.

That's where embeddings enter.

What is an embedding?

An embedding model converts text into a vector.

For example:

```
"How do I authenticate a user?"

                ↓
```

```
        Embedding Model

              ↓

[0.12, -0.81, 0.44, 0.09, ...]
```

Maybe 384 dimensions.

Or 768.

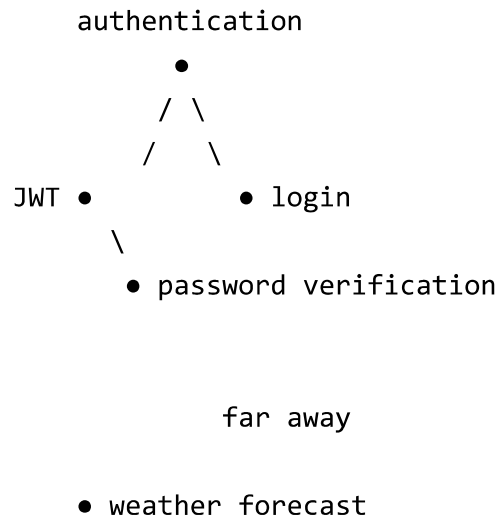
Or 1536.

The actual numbers aren't meaningful individually.

The important property is:

> **Similar meanings should produce nearby vectors.**

Conceptually:



So now retrieval becomes:

```
Question
   ↓
Embedding
   ↓
Vector
   ↓
Find nearby vectors
   ↓
Relevant code
```

This is **semantic search**.

🧠 Important interview understanding

If an interviewer asks:

> "Why do you need embeddings?"

Don't say:

> "Embeddings convert text into vectors."

That's technically correct but weak.

A better answer:

> "Keyword matching is limited when the query and relevant code use different terminology. Embeddings allow us to represent semantic meaning in a vector space, so we can retrieve conceptually related code even when there isn't much lexical overlap."

That demonstrates understanding.

Phase 3 — Vector databases

Now suppose your repository contains:

`100,000 chunks`

and every chunk has an embedding.

You can't manually compare everything every time.

We need efficient similarity search.

Enter:

- FAISS
- Qdrant

- Pinecone
- Weaviate
- Milvus
- etc.

For **our first version**, I actually don't want you to start with Pinecone.

We'll probably begin locally.

Why?

Because I want you to understand:

> **What is a vector database actually doing?**

rather than:

> "I put embeddings in Pinecone."

Phase 4 — Similarity

Suppose we have:

```
Query vector:
```

```
Q = [0.8, 0.2, 0.7]
```

and:

```
Code A = [0.7, 0.3, 0.6]
```

```
Code B = [-0.2, 0.9, -0.1]
```

Which is more similar?

We can calculate something such as **cosine similarity**.

Conceptually:

```
similarity(Q, A) → high
```

```
similarity(Q, B) → low
```

So:

```
Question
   ↓
Embedding
   ↓
Similarity search
   ↓
Top K chunks
```

That gives us our first working retrieval system.

Phase 5 — But there is a HUGE problem

This is where GitContext becomes interesting.

Suppose the developer asks:

> "Where is `PaymentProcessor` defined?"

Semantic search may work.

But suppose they ask:

> "Where is error code `PAY_4037` handled?"

Or:

> "Find references to `PaymentProcessor.process()`."

Now exact text matters.

Embeddings aren't always ideal for this.

So we introduce:

BM25 / keyword retrieval

Instead of asking:

"What text has similar meaning?"

we ask:

"What text contains important matching terms?"

Now we have two retrieval mechanisms.

Semantic

```
"What happens when authentication fails?"

            ↓

Find conceptually related code
```

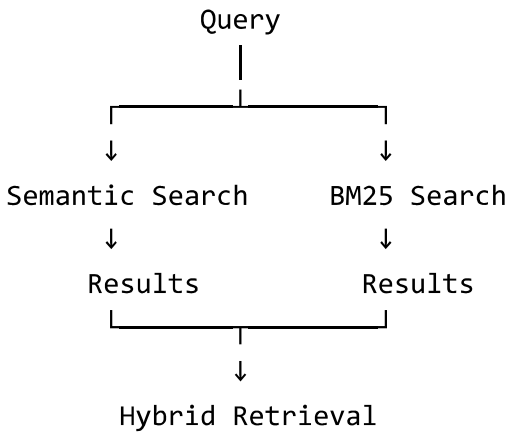
Keyword

```
"PAY_4037"

            ↓

Find exact occurrence
```

And then:



🔥 **Now you're doing actual retrieval engineering.**

Phase 6 — Intelligent code chunking

This is another place where your project becomes much more interesting.

A beginner RAG system does:

```
every 500 tokens → new chunk
```

That's convenient.

But code doesn't naturally work like that.

Imagine:

```
def process_payment():
    validate_user()
    check_balance()
    create_transaction()
    send_confirmation()
```

If you arbitrarily split it:

```
Chunk 1:
def process_payment():
    validate_user()
    check_balance()


Chunk 2:
create_transaction()
send_confirmation()
```

You've destroyed the semantic unit.

Instead:

```
Function = chunk
```

So our parser should understand:

```
file
 ├── class
 │     ├── method
 │     ├── method
 │     └── method
 │
```

```
├── function
└── function
```

Now you are learning something beyond RAG:

AST / code parsing

For languages such as Python, JavaScript, Java, etc., we can use parsers/tree-sitter-style approaches to understand code structure.

Phase 7 — Metadata

Suppose retrieval returns:

```
def authenticate_user(...)
```

That's useful.

But we want:

```
File:
backend/auth/login.py

Function:
authenticate_user()

Language:
Python

Module:
authentication

Commit:
a82f31

Author:
Rahul

Last modified:
2026-10-12
```

Now your system can answer questions such as:

> "Show me the authentication code modified in the last month."

That's why metadata matters.

Phase 8 — Reranking

Now imagine our hybrid search returns:

```
Top 20 results
```

But perhaps only 3 are actually useful.

So we add another stage:

```
Question
   ↓
Retriever
   ↓
20 candidates
   ↓
Reranker
   ↓
Top 5
   ↓
LLM
```

This is important.

Your first instinct might be:

> "Why not just retrieve the best 5?"

Because retrieval and reranking solve slightly different problems.

The first stage should be:

> **fast + broad**

The second stage:

> **slower + precise**

This is a classic production-system tradeoff.

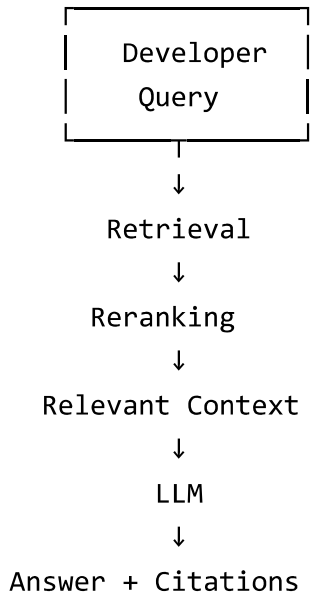
Phase 9 — The LLM

Only now do we bring in the LLM.

And this is important:

The LLM isn't the star.

The pipeline is:



The LLM's job is primarily:

> **Reason over the evidence we've retrieved.**

Not:

> "Remember the whole repository."

Phase 10 — Citations

This is where our system becomes much more useful.

Suppose it says:

> The payment validation occurs before transaction creation.

We should be able to show:

```
backend/payments/payment.py
lines 42–57
```

So the developer can verify the answer.

This is extremely important for reducing hallucination.

And now you can ask:

> "How accurate are those citations?"

That becomes an evaluation problem.

Phase 11 — Evaluation

This is where most student AI projects stop.

We won't.

We'll create a benchmark.

For example:

```
Question 1:
Where is JWT validation implemented?

Expected:
backend/auth/jwt.py

Question 2:
What happens when payment validation fails?
```

```
Expected:
backend/payments/payment.py

Question 3:
Which commit introduced refund validation?

Expected:
commit abc123
```

Then measure:

Recall@5

Did the correct document appear in the top 5?

Precision

How many retrieved results were actually relevant?

Answer correctness

Did the LLM produce the correct answer?

Citation accuracy

Did the answer cite the correct source?

Latency

How long did retrieval take?

Now you have **numbers**.

And numbers make your project defensible in interviews.

Phase 12 — Production problems

Once the basic system works, we're going to intentionally make it fail.

🔥 This is where I want your project to become exciting.

We'll investigate questions like:

What if the repository changes?

```
Version 1

login.py
   ↓
indexed

Developer modifies login.py

        ↓

Does our database know?
```

If not:

stale retrieval.

So we build incremental indexing.

What if retrieval fails?

Question:

> "Why does the payment gateway timeout after 30 seconds?"

But our system retrieves irrelevant code.

Should the LLM confidently hallucinate?

Absolutely not.

We need a mechanism to detect:

> "I don't have sufficient evidence."

What if the query is ambiguous?

> "How does auth work?"

Could refer to:

- frontend authentication
- backend authentication
- OAuth
- JWT
- database permissions

We can potentially ask clarifying questions or retrieve multiple perspectives.

What if the repository has 1 million lines?

Now:

- latency
- indexing time
- storage
- embedding cost
- retrieval quality

become engineering problems.

🗺️ Your learning roadmap

We're going to build this in stages.

Level 1 — AI fundamentals

You'll learn:

```
LLMs
↓
Tokens
↓
Embeddings
↓
Vector representations
↓
Similarity
↓
Cosine similarity
```

Goal: understand what the AI components are actually doing.

Level 2 — Build tiny retrieval system

We'll create:

```
Documents
   ↓
Chunks
   ↓
Embeddings
   ↓
Vector search
   ↓
Top-K
```

No fancy framework.

Goal: understand RAG mechanically.

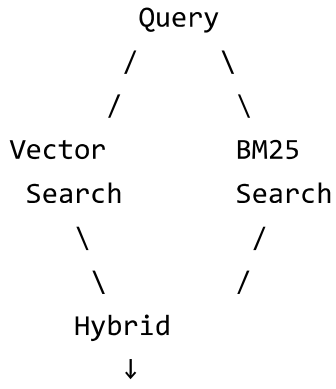
Level 3 — Turn it into code retrieval

```
Repository
   ↓
Code parser
   ↓
Function/class chunks
   ↓
```

```
Embeddings
   ↓
Retrieval
```

Goal: understand why code is different from normal documents.

Level 4 — Hybrid retrieval



Goal: understand retrieval quality.

Level 5 — Reranking

```
100 candidates
      ↓
Retriever
      ↓
20 candidates
      ↓
Reranker
      ↓
5 candidates
```

Goal: understand multi-stage retrieval.

Level 6 — LLM generation

```
Query
+
```

```
Retrieved context
        ↓
       LLM
        ↓
Answer
```

Goal: understand grounded generation.

Level 7 — Citations + confidence

```
Answer
 ↓
Evidence
 ↓
Citation
 ↓
Confidence
```

Goal: reduce hallucination.

Level 8 — Git intelligence

Now we ingest:

```
Code
README
Issues
Commits
PRs
```

And the system can answer:

> "Why was this function changed?"

That's when **GitContext really becomes GitContext.**

Level 9 — Evaluation

Build:

```
Evaluation Dataset
       ↓
Run system
       ↓
Metrics
       ↓
Failure analysis
       ↓
Improve retrieval
       ↓
Run again
```

This is **AI engineering thinking**.

Level 10 — Production

Finally:

```
FastAPI
   ↓
Caching
   ↓
Async processing
   ↓
Incremental indexing
   ↓
Logging
   ↓
Monitoring
   ↓
Deployment
```

Now you have an actual system.

🎯 And here's how I want to teach you

For every new concept, we'll follow this cycle:

1. Problem

Why do we need this?

2. Intuition

Explain it in normal human language.

3. Mathematics / internals

Only as much as you need to truly understand it.

4. Tiny experiment

You implement a miniature version.

5. GitContext implementation

We put it into the actual project.

6. Break it

I give you cases where it fails.

7. Fix it

You reason about the solution.

8. Interview explanation

You learn how to explain what you built.

🚀 And I don't want you starting with LangChain

This is deliberate.

Frameworks are useful **after** you understand what they're hiding.

If you start with:

```
chain = RetrievalQA(...)
```

you may successfully build RAG without knowing what RAG actually does.

Instead, we'll initially build:

```
Embedding
      ↓
Vector
      ↓
Similarity
      ↓
Top-K
      ↓
Context
      ↓
LLM
```

ourselves.

Then later, when we encounter a framework, you'll be able to say:

> "I understand what this abstraction is doing underneath."

That is the knowledge you're after.

🔥 Your first mission

Before we write GitContext, I want you to understand **one concept deeply: embeddings**.

Don't worry about:

- GitHub API
- Qdrant
- LangChain
- FastAPI
- React
- deployment
- agents

yet.

We're going to start with a tiny experiment where **you will actually see why embeddings are useful**.

We'll take sentences like:

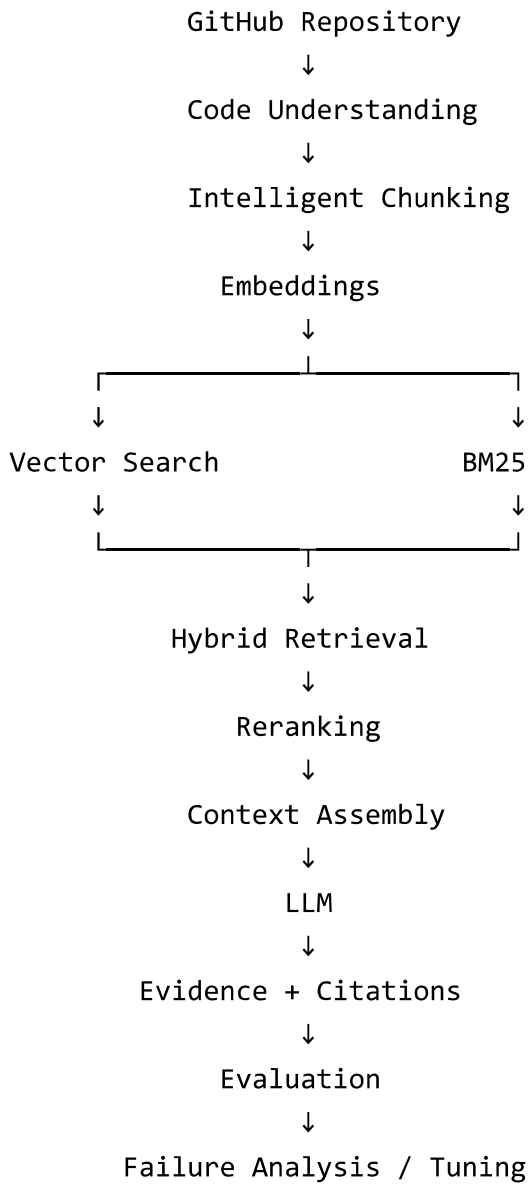
```
"I want to reset my password"
"My password needs to be changed"
"How do I authenticate with JWT?"
"What's the weather today?"
```

and turn them into vectors.

Then we'll calculate similarity ourselves and **watch semantic search happen**.

Once you genuinely understand that, we'll move to vector databases.

That way, by the time we eventually build the full GitContext system, you'll know what every box in this architecture is doing:



That's the journey I'd take you through. And by the end, you shouldn't just be able to say *"I built a RAG project."* You should be able to explain **why each component exists, what happens when it fails, how you measured it, and what trade-offs you made.**

You said:

yes let's goo

ChatGPT said:

LET'S GOOOO 🚀🔥

We're going to do this like you're actually becoming an **AI engineer**, not like you're following a YouTube tutorial.

🧠 GitContext — Lesson 1

What the hell is an embedding?

Before touching GitHub, RAG, vector databases, or LLMs, you need to understand this one idea.

---

1. First, forget AI for a minute

Imagine I give you these sentences:

```
A. "I want to reset my password."

B. "How can I change my password?"

C. "The weather is very nice today."

D. "How do I authenticate a user?"
```

As a human, you immediately know:

```
A ↔ B     very similar

A ↔ D     somewhat related

A ↔ C     unrelated
```

A computer doesn't inherently understand that.

To a computer, they're just sequences of characters.

So we need a way to represent **meaning** numerically.

That's what embeddings do.

---

2. Embedding = meaning represented as numbers

An embedding model takes:

```
"I want to reset my password."
```

and produces something conceptually like:

```
[0.21, -0.73, 0.45, 0.18, ...]
```

Maybe there are hundreds of numbers.

For example, imagine only **3 dimensions** for simplicity:

```
"I want to reset my password"
        ↓
      [0.8, 0.7, 0.9]

"How can I change my password?"
        ↓
      [0.82, 0.68, 0.91]

"The weather is nice"
        ↓
      [-0.2, 0.1, -0.7]
```

Now something interesting happens.

The first two vectors are close together.

The weather vector is far away.

So we can mathematically ask:

> **How similar are these two pieces of text?**

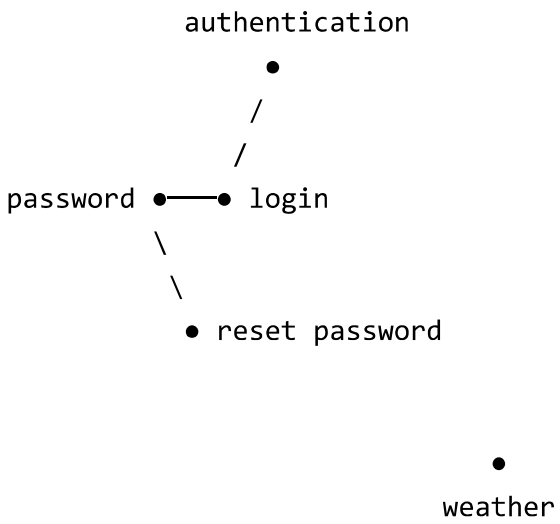
---

3. Think of embeddings as coordinates

This is the easiest mental model.

Imagine a huge map.

Every piece of text gets a location.



Similar concepts end up near each other.

Different concepts end up farther apart.

This space is called an **embedding space**.

---

4. But who decides where things go?

The **embedding model**.

You give it:

`"How can I change my password?"`

and it generates a vector.

You give it:

`"I forgot my password."`

and it generates another vector.

Because the model has learned language relationships from training, those vectors should be relatively close.

Important:

The model isn't literally writing:

```
password = 0.8
authentication = 0.7
weather = -0.2
```

The dimensions don't have simple human-readable meanings.

The **overall position in the high-dimensional space** captures useful semantic relationships.

---

5. Now comes the REALLY important part

Suppose our user asks:

> **"How do I reset my password?"**

Our repository contains:

```
A:
"Password reset is handled by the forgotPassword() function."

B:
"The JWT token is generated by authenticateUser()."

C:
"Weather API requests are processed every 30 minutes."
```

We turn everything into embeddings.

```
Query
 ↓
Embedding
 ↓
Q

A
 ↓
Embedding
 ↓
```

```
A'

B
 ↓
Embedding
 ↓
B'

C
 ↓
Embedding
 ↓
C'
```

Then:

```
similarity(Q, A') → 0.91
similarity(Q, B') → 0.62
similarity(Q, C') → 0.08
```

So we retrieve:

```
A
```

🔥 **This is the foundation of semantic retrieval.**

---

6. How do we measure "close"?

This is where mathematics enters.

One popular method is:

Cosine similarity

Don't panic. 😄

The idea is simple.

Given two vectors:

$$A = [a_1, a_2, a_3, ...]$$

$$B = [b_1, b_2, b_3, ...]$$

cosine similarity measures the angle between them:

$$\text{cosine similarity} = \frac{A \cdot B}{\|A\|\|B\|}$$

The result is generally between:

```
-1 ———————— 0 ———————— +1
```

Conceptually:

```
+1 → extremely similar direction

 0 → unrelated

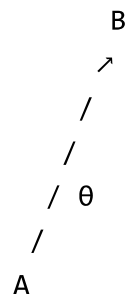
-1 → opposite direction
```

For many modern text embeddings, you'll commonly see useful similarity comparisons concentrated in a more limited positive range, so don't obsess over the exact numerical interpretation yet.

---

7. Let's visualize it

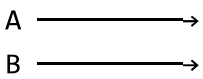
Imagine:



Cosine similarity is essentially asking:

> **How aligned are these vectors?**

Small angle:



```
Very similar
```

Large angle:

```
A ──────────→

          ↗ B

Less similar
```

---

8. Here's where GitContext starts becoming powerful

Suppose your code contains:

```
def verify_jwt(token):
    decoded = jwt.decode(token, SECRET_KEY)
    return decoded
```

The developer asks:

> **"Where do we authenticate users?"**

Notice something interesting.

The query says:

```
authenticate users
```

The code says:

```
verify_jwt
```

They don't use exactly the same words.

Keyword search might struggle.

But an embedding model can potentially understand that:

```
authenticate
```

and

`verify JWT`

are semantically related.

That's why embeddings are so valuable for code search.

---

9. BUT WAIT... 🚨

This is the first important weakness.

Suppose the developer asks:

> **"Where is error code `PAY_4037` handled?"**

Imagine the code:

```
if error_code == "PAY_4037":
    retry_payment()
```

This is an **exact identifier**.

We don't necessarily want semantic similarity.

We want:

`PAY_4037`

to exactly match:

`PAY_4037`

And that's why later we're going to learn:

BM25

We'll combine:

```
Semantic Search
       +
Keyword Search
```

```
      ↓
Hybrid Retrieval
```

This is one of the first important design decisions in GitContext.

🧪 YOUR FIRST EXPERIMENT

Now I want you to **actually see embeddings instead of just hearing about them.**

We're going to make a tiny Python project.

Don't create GitContext yet.

Create:

```
embedding-experiment/
└── experiment.py
```

But **don't write the code yet**.

First I want you to answer three questions yourself:

Q1.

If we have:

```
"I forgot my password"
"How can I reset my password?"
"The stock market fell today"
```

Which two should have the most similar embeddings?

Q2.

If:

```
Query embedding = Q

Document embeddings = A, B, C, D, E
```

and their similarity scores are:

```
A = 0.21
B = 0.89
C = 0.34
D = 0.76
E = 0.12
```

Which documents should our retriever return if `K = 2` ?

Q3.

Suppose the query is:

```
"Where is PAY_4037 handled?"
```

and the correct code contains exactly:

`PAY_4037`

but the embedding similarity is unexpectedly low.

What kind of retrieval mechanism might help us here?

Reply with your answers **in your own words**.

Don't Google them.

Don't worry about being wrong.

This is our first little test because I want to see whether the concept has actually clicked before we touch code. 😈🧠

You said:

ok Q1 - "I forgot my password"
"How can I reset my password?", Q2 - B, D , Q3 - BM25

ChatGPT said:

YESSS 🔥 **3/3.** And more importantly, your reasoning is correct.

You already have the core idea of embeddings + retrieval. Now let's push one level deeper.

🧠 Lesson 1.5 — Let's actually create embeddings

We're going to do a tiny experiment before GitContext.

The goal is:

> **You should see semantic similarity happening with your own eyes.**

Step 1 — What are we going to build?

Very simple:

```
Sentence
   ↓
Embedding Model
   ↓
Vector
   ↓
Cosine Similarity
   ↓
Similarity Score
```

We'll give it:

```
1. "I forgot my password"
2. "How can I reset my password?"
3. "The weather is beautiful today"
4. "How do I authenticate a user?"
```

Then we'll compare them.

We're expecting something approximately like:

```
                          Similarity

Password ↔ Reset password       HIGH
Password ↔ Authenticate         MEDIUM/HIGH
Password ↔ Weather              LOW
```

The exact numbers are not important.

The relationship is.

Step 2 — One important decision

There are many embedding models.

For learning, we're going to use a **local open-source sentence embedding model** rather than immediately using a paid API.

Why?

Because I want you to understand:

> "I send text → model converts it into a vector."

without worrying about API keys or costs.

We'll use the Python ecosystem around **Sentence Transformers**.

Conceptually:

```
Sentence Transformers
        ↓
Embedding model
        ↓
Vector
```

Step 3 — Create the environment

Open your terminal.

Create a folder:

```
mkdir embedding-experiment
cd embedding-experiment
```

Create a virtual environment:

```
python -m venv .venv
```

Activate it on Windows:

```
.venv\Scripts\activate
```

You should see something like:

```
(.venv) PS C:\...\embedding-experiment>
```

Then install:

```
pip install sentence-transformers
```

Step 4 — Your first AI code

Create:

```
experiment.py
```

And put this in it:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

sentences = [
    "I forgot my password",
    "How can I reset my password?",
    "The weather is beautiful today",
    "How do I authenticate a user?"
]

embeddings = model.encode(sentences)

for sentence, embedding in zip(sentences, embeddings):
    print(sentence)
    print("Vector length:", len(embedding))
    print("First 5 values:", embedding[:5])
    print()
```

Then run:

```
python experiment.py
```

🛑 Before you continue

I don't want you to blindly paste this.

Let's understand **every important line**.

This:

```
model = SentenceTransformer("all-MiniLM-L6-v2")
```

means:

> Load a pretrained model capable of converting sentences into numerical embeddings.

Then:

```
embeddings = model.encode(sentences)
```

means:

> Take every sentence and convert it into an embedding vector.

So if we have:

```
4 sentences
```

we'll get:

```
4 vectors
```

And `all-MiniLM-L6-v2` produces vectors of a particular fixed dimensionality.

You don't need to memorize that dimension yet.

What's important is:

```
text → vector
```

🧠 Now comes the fun part

We have vectors.

But we haven't answered the important question:

> **How similar are the vectors?**

So we're going to calculate cosine similarity.

Add:

```
from sentence_transformers.util import cos_sim
```

Then:

```
similarity = cos_sim(embeddings, embeddings)
```

and:

```
print(similarity)
```

Now run it.

You'll get something resembling a matrix:

```
                  Sentence 1   Sentence 2   Sentence 3   Sentence 4

Sentence 1            1.00         0.XX         0.XX         0.XX
Sentence 2            0.XX         1.00         0.XX         0.XX
Sentence 3            0.XX         0.XX         1.00         0.XX
Sentence 4            0.XX         0.XX         0.XX         1.00
```

Don't worry about the exact numbers.

🤯 Look at the diagonal

Why is this:

```
Sentence 1 ↔ Sentence 1 = 1
```

?

Because a vector is perfectly similar to itself.

Same for:

```
Sentence 2 ↔ Sentence 2
Sentence 3 ↔ Sentence 3
Sentence 4 ↔ Sentence 4
```

🔎 Now look at the interesting cells

You should inspect:

```
Sentence 1 ↔ Sentence 2
```

These are:

> "I forgot my password"

and

> "How can I reset my password?"

That should be relatively high.

Now:

```
Sentence 1 ↔ Sentence 3
```

That's:

> password

vs.

> weather

That should be substantially lower.

🚨 Here's the first AI engineering lesson

The model isn't saying:

> "Sentence 1 is 83% correct."

It is giving us a **similarity signal**.

And that's incredibly useful.

Because now we can do:

```
User Query
     ↓
Embedding
     ↓
Compare against document embeddings
     ↓
Rank by similarity
     ↓
Return top K
```

Congratulations.

You've just built the conceptual core of a **semantic search engine**.

And semantic search is one of the foundations of our GitContext RAG system.

🧩 Your challenge

Don't just run the original four sentences.

Change them.

Try:

```
sentences = [
    "How do I log into my account?",
    "How can I authenticate a user?",
    "The user needs to verify their identity.",
    "The football match starts at 8 PM."
]
```

Before running it, **predict**:

Which pair will have the highest similarity?

And:

Which pair will have the lowest similarity?

Then run it and see whether your intuition matches the model.

One more challenge 😈

Add:

```
"JWT tokens are used for authentication."
```

Now predict which sentence it will be closest to.

Don't worry if the result surprises you.

If the result surprises you, that's actually better.

That's how we'll start discovering that AI systems don't behave exactly like human intuition—and that's where the interesting engineering begins.

You said:

next

ChatGPT said:

Absolutely. 🚀 Now we move from "**I can generate embeddings**" to "**I can use embeddings to retrieve information.**"

🧠 Lesson 2 — Build Semantic Search From Scratch

This is the moment where embeddings become useful.

We currently have:

```
Text
 ↓
```

```
Embedding
 ↓
Vector
```

But our actual goal is:

```
User question
      ↓
Find the most relevant information
      ↓
Give it to the LLM
```

So let's build that retrieval layer.

---

1. Imagine our tiny knowledge base

Suppose GitContext has these pieces of code:

```
def login_user(username, password):
    user = find_user(username)
    if verify_password(password, user.password_hash):
        return create_session(user)

def verify_password(password, password_hash):
    return bcrypt.checkpw(password, password_hash)

def create_session(user):
    token = generate_jwt(user.id)
    return token

def send_email(to, subject, body):
    smtp.send(to, subject, body)
```

Now developer asks:

> **"How is a user's password checked?"**

Our system should find:

```
def verify_password(password, password_hash):
    return bcrypt.checkpw(password, password_hash)
```

The interesting thing is that the question doesn't contain:

```
verify_password
bcrypt
password_hash
```

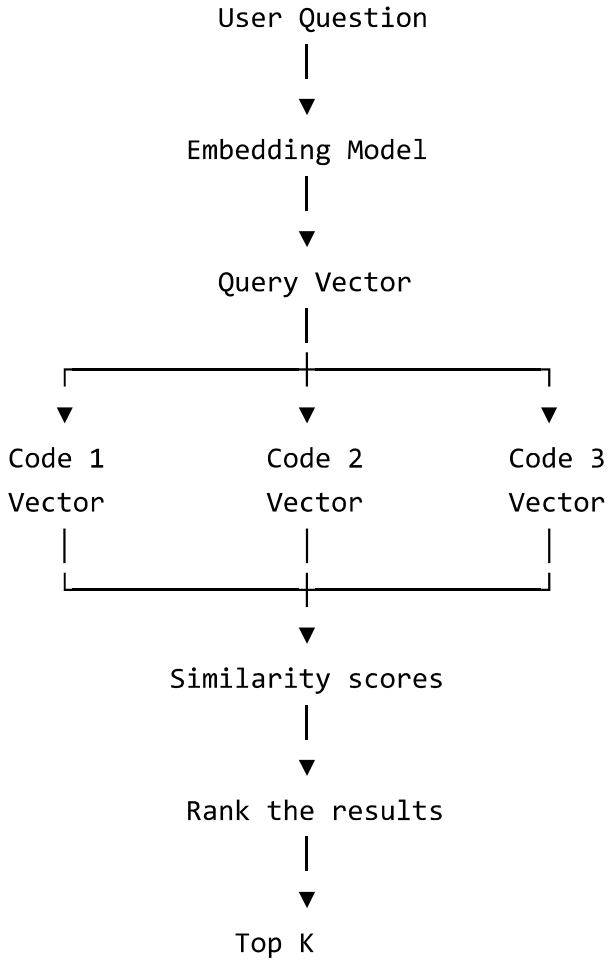
exactly.

That's where semantic search helps.

---

2. The retrieval pipeline

We do this:



This is the **retrieval** part of RAG.

---

3. Let's make the idea concrete

Suppose we have 4 documents:

```
D1 = login code
D2 = password verification code
D3 = JWT generation code
D4 = email code
```

We embed all four.

Then embed the query:

```
Q = "How is a user's password checked?"
```

Imagine the similarity scores are:

```
D1 → 0.71
D2 → 0.91
D3 → 0.43
D4 → 0.08
```

We sort them:

```
D2 → 0.91  ← #1
D1 → 0.71  ← #2
D3 → 0.43  ← #3
D4 → 0.08  ← #4
```

If:

```
K = 2
```

we return:

```
D2
D1
```

That's **Top-K retrieval**.

---

4. Let's code it

Create a new file:

semantic_search.py

Use:

```
from sentence_transformers import SentenceTransformer
from sentence_transformers.util import cos_sim

model = SentenceTransformer("all-MiniLM-L6-v2")

documents = [
    """
    def login_user(username, password):
        user = find_user(username)
        if verify_password(password, user.password_hash):
            return create_session(user)
    """,

    """
    def verify_password(password, password_hash):
        return bcrypt.checkpw(password, password_hash)
    """,

    """
    def create_session(user):
        token = generate_jwt(user.id)
        return token
    """,

    """
    def send_email(to, subject, body):
        smtp.send(to, subject, body)
    """
]

query = "How is a user's password checked?"

document_embeddings = model.encode(documents)
query_embedding = model.encode(query)

similarities = cos_sim(query_embedding, document_embeddings)[0]

results = []
```

```
for i, score in enumerate(similarities):
    results.append((i, float(score)))

results.sort(key=lambda x: x[1], reverse=True)

for index, score in results:
    print(f"Score: {score:.4f}")
    print(documents[index])
    print("-" * 50)
```

Run it.

---

5. Understand what we just built

Don't focus on the Python syntax yet.

The important architecture is:

```
documents
    ↓
embedding model
    ↓
document embeddings
```

Separately:

```
query
  ↓
embedding model
  ↓
query embedding
```

Then:

```
query embedding
       ↓
compare
       ↓
document embeddings
       ↓
similarity scores
       ↓
```

```
sort
      ↓
top results
```

That is our first **retriever**.

🧠 The important distinction

We now have two different things:

Embedding model

Converts:

```
text → vector
```

Retriever

Uses those vectors to answer:

```
Which documents are most relevant to this query?
```

This distinction will become very important later.

---

6. Now let's ask a harder question

Imagine our repository has:

```
10 files
```

Our code works.

Then:

```
10,000 files
```

We generate 10,000 embeddings.

For every query, we'd theoretically compare the query against all 10,000.

That's possible for small datasets.

But imagine:

```
10 million chunks
```

Now things become interesting.

We need efficient vector search.

And that's where:

🗄️ Vector databases

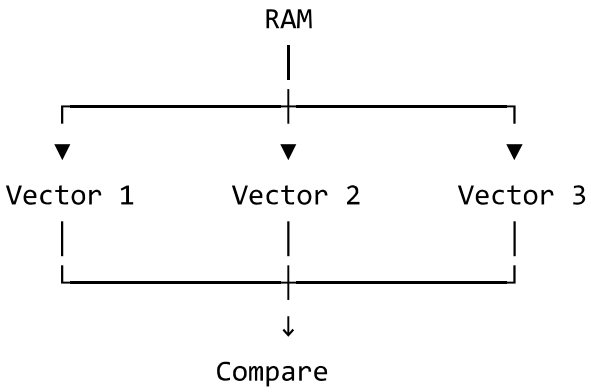
come in.

---

7. But don't jump to Qdrant yet

I want you to understand **why** we need one.

Our current system is basically:



We're manually holding everything.

A proper vector database gives us things like:

- persistent storage
- efficient similarity search
- metadata filtering
- indexing
- scalability
- updates/deletions

So instead of:

```
"compare against everything manually"
```

we can ask:

```
Vector DB:

"Give me the 5 vectors most similar to this query."
```

---

8. But here's an important concept

A vector database is **not AI magic**.

It doesn't understand your question.

It doesn't "know" what your code means.

Usually:

```
Embedding model
        ↓
turns meaning into vectors

Vector database
        ↓
stores/searches those vectors
```

Keep those responsibilities separate.

That's an excellent interview point.

If an interviewer asks:

> "**What does Qdrant do in your architecture?**"

Don't say:

> "Qdrant understands the code."

Say:

> "The embedding model converts code and queries into vector representations. Qdrant stores those vectors and performs efficient similarity search, along with metadata filtering."

Much stronger.

9. One more problem before we use a vector DB

Our current documents are already nicely separated.

Real repositories aren't.

Imagine:

`payment.py`

contains:

```
class PaymentService:

    def validate_user(...):
        ...

    def check_balance(...):
        ...

    def create_transaction(...):
        ...

    def refund_payment(...):
        ...

    def send_notification(...):
        ...
```

Should we make the entire file **one embedding**?

Probably not.

Why?

Suppose the user asks:

> "How does refunding work?"

The embedding represents a huge mixture of:

```
validation
balance
```

```
transaction
refund
notification
```

The relevant signal could get diluted.

So we need to split code intelligently.

🔥 This leads to our next major concept:

Chunking

For normal documents:

```
paragraph → chunk
```

For code:

```
file
 ├── class
 │    ├── method
 │    ├── method
 │    └── method
 │
 ├── function
 └── function
```

So instead of:

```
payment.py → 1 chunk
```

we might have:

```
PaymentService.validate_user()     → chunk 1
PaymentService.check_balance()     → chunk 2
PaymentService.create_transaction() → chunk 3
PaymentService.refund_payment()    → chunk 4
PaymentService.send_notification() → chunk 5
```

Now retrieval can return **exactly the relevant function**.

And this is where GitContext starts becoming more than a generic RAG chatbot.

🧩 Your mini challenge

Before we start chunking, think about this repository:

```
backend/
├── auth.py
├── payments.py
├── users.py
└── email.py
```

`payments.py` contains:

```
def validate_payment():
    ...

def calculate_tax():
    ...

def process_payment():
    ...

def refund_payment():
    ...
```

A developer asks:

> **"How do refunds work?"**

Question 1

Would you prefer to embed:

A. The entire `payments.py` file as one vector

or

B. Each function separately?

Why?

Question 2

Suppose we split every function separately.

What useful information might we accidentally **lose**?

Think carefully. 👀

For example, `refund_payment()` might depend on:

```
validate_payment()
calculate_tax()
process_payment()
```

So there's a tradeoff.

Don't look for the "correct" answer online. I want you to reason about it.

Once you answer these two, we'll tackle **code chunking + context preservation**, and then we'll build our first actual **mini-RAG system**.

You said:

no can u first explain the code for the semantic _search

ChatGPT said:

Absolutely. 👍 We should **slow down here**. If you don't understand the semantic search code, moving ahead will just turn this into copy-paste learning.

Let's take the `semantic_search.py` **line by line**, from zero.

🧠 First: What are we trying to make?

Our goal is very simple:

> Given a question, find which piece of text is most related to that question.

We have:

```
Documents
   ↓
Convert documents → vectors
```

```
Question
   ↓
Convert question → vector

Compare question vector with document vectors
   ↓
Rank them
   ↓
Most similar document = most relevant
```

Now let's understand how the code does exactly that.

---

1. Import the model

```
from sentence_transformers import SentenceTransformer
```

We're importing the `SentenceTransformer` class from the `sentence_transformers` library.

Remember our previous experiment?

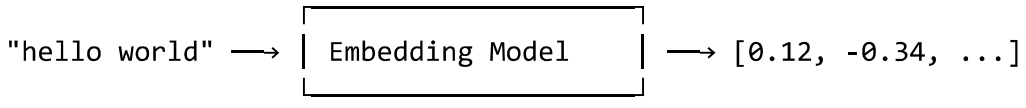
We did:

```
model = SentenceTransformer("all-MiniLM-L6-v2")
```

The model's job is:

```
text → embedding
```

Think of `model` as a machine:



---

2. Import cosine similarity

```
from sentence_transformers.util import cos_sim
```

We need a way to compare vectors.

Remember:

```
Vector A
   ↕
Vector B
```

We want:

> How similar are these two vectors?

`cos_sim()` calculates **cosine similarity**.

So conceptually:

```
cos_sim(A, B)
```

means:

> "Give me a similarity score between A and B."

---

3. Load the embedding model

```
model = SentenceTransformer("all-MiniLM-L6-v2")
```

This loads a pretrained embedding model.

We're saying:

> "Use `all-MiniLM-L6-v2` as our text → vector converter."

Important distinction:

We're **not training the model**.

We're using a model that has already been trained.

So:

```
Our code
   ↓
pretrained embedding model
```

```
   ↓
vectors
```

Later, we'll discuss what exactly "pretrained" means.

---

4. Create our documents

```
documents = [
    """
    def login_user(username, password):
        user = find_user(username)
        if verify_password(password, user.password_hash):
            return create_session(user)
    """,

    """
    def verify_password(password, password_hash):
        return bcrypt.checkpw(password, password_hash)
    """,

    """
    def create_session(user):
        token = generate_jwt(user.id)
        return token
    """,

    """
    def send_email(to, subject, body):
        smtp.send(to, subject, body)
    """
]
```

This is just a Python **list of strings**.

Think of it as our tiny database.

We have four documents:

```
Document 0 → login_user()
Document 1 → verify_password()
Document 2 → create_session()
Document 3 → send_email()
```

The numbers are important because Python lists use indexes:

```
documents[0] → login_user
documents[1] → verify_password
documents[2] → create_session
documents[3] → send_email
```

---

5. Create the user's query

```
query = "How is a user's password checked?"
```

This is what our imaginary developer typed.

So our problem is:

> Among these four documents, which one answers this question?

---

6. Convert ALL documents into embeddings

This line is extremely important:

```
document_embeddings = model.encode(documents)
```

Let's break it down.

We give:

```
documents
```

to:

```
model.encode()
```

The model processes every document.

Conceptually:

```
Document 0
     ↓
Embedding model
     ↓
```

```
Vector 0

Document 1
     ↓
Embedding model
     ↓
Vector 1

Document 2
     ↓
Embedding model
     ↓
Vector 2

Document 3
     ↓
Embedding model
     ↓
Vector 3
```

So:

`document_embeddings`

contains something conceptually like:

```
[
    [0.12, -0.45, 0.73, ...],   ← Document 0
    [0.21, -0.31, 0.81, ...],   ← Document 1
    [0.04,  0.92, 0.11, ...],   ← Document 2
    [-0.7, 0.12, 0.33, ...]     ← Document 3
]
```

The actual vectors are much longer.

---

7. Convert the QUESTION into an embedding

Next:

```
query_embedding = model.encode(query)
```

This does exactly the same thing.

But now we're encoding **one question**.

```
"How is a user's password checked?"
                ↓
          embedding model
                ↓
         query vector
```

Conceptually:

```
query_embedding =
[0.19, -0.40, 0.79, ...]
```

Now we have:

```
Documents → vectors
Question  → vector
```

We're ready to compare them.

---

8. Calculate similarities

This is the most important line:

```
similarities = cos_sim(query_embedding, document_embeddings)[0]
```

Let's ignore `[0]` for a moment.

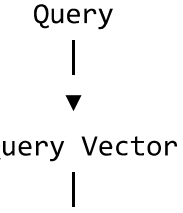
The important part is:

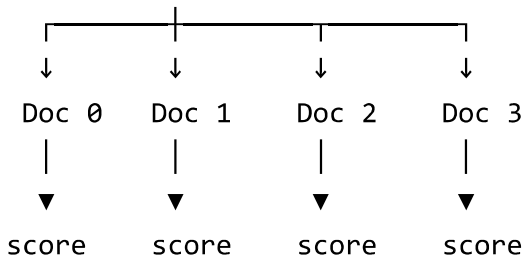
```
cos_sim(query_embedding, document_embeddings)
```

We're asking:

> Compare the query vector against every document vector.

So conceptually:





We might get something like:

```
[0.72, 0.91, 0.42, 0.08]
```

Meaning:

```
Document 0 → 0.72
Document 1 → 0.91
Document 2 → 0.42
Document 3 → 0.08
```

Therefore:

```
Document 1 is most similar.
```

And Document 1 is:

```
def verify_password(password, password_hash):
    return bcrypt.checkpw(password, password_hash)
```

Exactly what we wanted.

---

9. But what's `[0]` doing?

You might have noticed:

```
cos_sim(query_embedding, document_embeddings)[0]
```

Why `[0]` ?

Because the similarity function returns a **matrix-like result**.

Conceptually, we're comparing:

```
1 query

against
```

```
4 documents
```

So the result is shaped approximately like:

```
[
    [0.72, 0.91, 0.42, 0.08]
]
```

That's a 1 × 4 structure.

We only care about the row:

```
[0.72, 0.91, 0.42, 0.08]
```

So:

```
[0]
```

selects that first row.

This is a small Python/tensor detail; don't worry if the exact array dimensions aren't intuitive yet.

---

10. Now we need to connect scores to documents

We currently have two separate things:

Documents

```
Index 0 → login_user
Index 1 → verify_password
Index 2 → create_session
Index 3 → send_email
```

Scores

```
Index 0 → 0.72
Index 1 → 0.91
Index 2 → 0.42
Index 3 → 0.08
```

We need to combine them.

That's what this does:

```
results = []
```

We're creating an empty list.

Then:

```
for i, score in enumerate(similarities):
    results.append((i, float(score)))
```

Let's understand `enumerate()`.

Suppose:

```
similarities = [0.72, 0.91, 0.42, 0.08]
```

Then:

```
enumerate(similarities)
```

gives us:

```
i = 0, score = 0.72
i = 1, score = 0.91
i = 2, score = 0.42
i = 3, score = 0.08
```

So we're building:

```
results = [
    (0, 0.72),
    (1, 0.91),
    (2, 0.42),
    (3, 0.08)
]
```

Why keep the index?

Because we need to remember **which document produced the score**.

---

11. Why `float(score)`?

`float(score)`

The similarity value may be stored as a tensor/numerical type rather than a normal Python `float`.

We're simply converting it into an ordinary Python number.

So:

```
tensor value
    ↓
Python float
```

Not an AI concept—just a programming convenience.

---

12. Now comes ranking

We have:

```
results = [
    (0, 0.72),
    (1, 0.91),
    (2, 0.42),
    (3, 0.08)
]
```

But we want highest similarity first.

So:

```
results.sort(key=lambda x: x[1], reverse=True)
```

This line looks scary, but it's actually simple.

---

x

Each item is:

```
(index, score)
```

For example:

```
(1, 0.91)
```

So:

```
x[0]
```

means:

`index`

and:

`x[1]`

means:

`score`

We want to sort by the score.

Therefore:

`key=lambda x: x[1]`

means:

> "Use the second element—the similarity score—as the sorting key."

And:

`reverse=True`

means:

> Highest → lowest.

So:

`Before:`

```
(0, 0.72)
(1, 0.91)
(2, 0.42)
(3, 0.08)
```

`After:`

```
(1, 0.91)
(0, 0.72)
```

```
(2, 0.42)
(3, 0.08)
```

---

13. Finally, print the results

`for index, score in results:`

We're going through:

```
(1, 0.91)
(0, 0.72)
(2, 0.42)
(3, 0.08)
```

Then:

`print(f"Score: {score:.4f}")`

prints something like:

`Score: 0.9100`

The `.4f` just means:

> Display 4 digits after the decimal point.

Then:

`print(documents[index])`

This is where everything connects.

If:

`index = 1`

then:

`documents[1]`

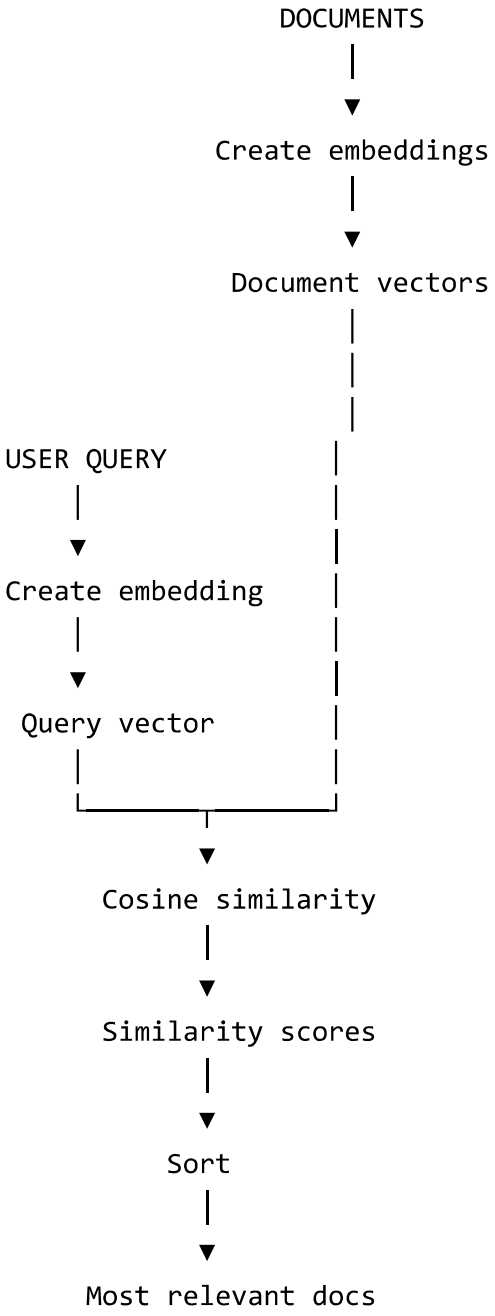
gives:

```
def verify_password(password, password_hash):
    return bcrypt.checkpw(password, password_hash)
```

🎯 So the entire program is basically this

Forget the Python syntax for a moment.

The algorithm is:



That's semantic search.

🧠 And THIS is the part I want you to remember

We are not really interested in this:

```
cos_sim(...)
```

as a random function.

We're interested in the **idea**:

> **Represent both the query and documents in the same vector space, then use distance/similarity in that space to retrieve semantically relevant information.**

That sentence is something you should eventually be able to explain in an interview.

🚨 One limitation of our current implementation

There's something very inefficient about what we just built.

Imagine:

```
10,000 documents
```

Every time a user asks a question, we're essentially doing:

```
Query
 ↓
Compare against
 ↓
10,000 vectors
 ↓
Sort 10,000 scores
```

For our tiny experiment: **completely fine.**

For a huge production system: **not ideal.**

And this brings us naturally to our next topic:

🗄️ Vector databases & Approximate Nearest Neighbor Search

We'll understand **why Qdrant/FAISS/etc. exist**, what they actually do, and why you shouldn't blindly say "*I used a vector database*" in an interview.

But before that, I want to make sure **semantic search itself has clicked**.

Quick check — answer these 3:

1. Why do we create embeddings for both the documents **and** the query?

2. Why can't we just compare the raw text directly to determine semantic similarity?

3. If we have 100 documents and the query has the highest similarity with document 73, what does `73` represent in our Python code?

You said:

1 selection

so there is a tradeoff defintely i think we can now divide here function only and then retriev the top-k instead of one so that the accidental loss might be reduced

ChatGPT said:

Exactly. 🔥 **You're thinking in the right direction.** But there's one more subtle issue we need to understand.

Your idea is:

> "Instead of embedding the whole file, split it into functions and retrieve Top-K functions."

That's a very reasonable first design.

---

1. Why your idea works

Suppose `payments.py` has:

```
validate_payment()
calculate_tax()
```

```
process_payment()
refund_payment()
send_notification()
```

Instead of:

```
payments.py
     ↓
one huge embedding
```

we do:

```
validate_payment()      → vector 1
calculate_tax()         → vector 2
process_payment()       → vector 3
refund_payment()        → vector 4
send_notification()     → vector 5
```

Then the user asks:

> "How does refunding work?"

Our semantic search might return:

```
1. refund_payment()       0.92
2. process_payment()      0.78
3. validate_payment()     0.71
```

Now the LLM gets those 3 pieces.

That's much better than giving it the entire file.

So yes:

> **Function-level chunking + Top-K retrieval reduces the risk of irrelevant code overwhelming the context.**

---

2. But here's the interesting problem 👀

Suppose:

```
def refund_payment(transaction_id):
    transaction = get_transaction(transaction_id)

    if not validate_payment(transaction):
        return False

    amount = calculate_refund(transaction)

    return process_refund(transaction, amount)
```

We retrieve:

`refund_payment()`

Great.

But suppose the implementation of:

`validate_payment()`

is somewhere else:

`payment_validation.py`

and:

```
def validate_payment(transaction):
    ...
```

The LLM may receive:

`refund_payment()`

but **not**:

`validate_payment()`

Now we have a new problem.

The chunk itself is relevant, but **its dependencies aren't present**.

---

3. This is what "context preservation" means

We don't just want:

> "Find the most similar chunks."

We want:

> **"Find the most relevant chunks while preserving enough context for the model to understand them."**

That's a much deeper problem.

Think about a textbook.

You wouldn't necessarily answer a question using only one sentence.

You might need:

```
Sentence
   ↓
Previous explanation
   ↓
Definition
   ↓
Example
```

Code is similar.

A function may depend on:

```
imports
global variables
parent class
other functions
configuration
database models
```

So blindly splitting everything into isolated functions can lose relationships.

---

4. This gives us several possible strategies

Strategy A — File-level chunks

```
payments.py → one chunk
```

Pros:

- lots of context

Cons:

- huge
- noisy
- poor retrieval precision

Strategy B — Function-level chunks

```
refund_payment() → chunk
validate_payment() → chunk
```

Pros:

- precise
- meaningful semantic units
- easier retrieval

Cons:

- dependencies can be missing

Strategy C — Function + surrounding context

For example:

```
refund_payment()

+ class name
+ imports
+ relevant comments
+ signatures
```

This gives us something like:

```
File: payments.py
Class: PaymentService

Function: refund_payment
```

```
Dependencies:
    validate_payment
    calculate_refund
    process_refund

Code:
    ...
```

Much richer.

---

5. And then we can combine retrieval with relationships

This is where I want our GitContext project to eventually become more interesting than a normal RAG system.

Suppose semantic retrieval gives us:

```
refund_payment()
```

Then we inspect its relationships:

```
refund_payment()
        │
        ├──────→ validate_payment()
        │
        ├──────→ calculate_refund()
        │
        └──────→ process_refund()
```

We could retrieve those related functions too.

So instead of:

```
Query
 ↓
Top 3 chunks
```

we could eventually have:

```
Query
 ↓
```

```
Top relevant chunk
 ↓
Understand its dependencies
 ↓
Retrieve relevant dependencies
 ↓
Build context
 ↓
LLM
```

🔥 Now we're moving toward **code-aware retrieval**, rather than generic text RAG.

---

6. But don't jump there yet

There's a very important engineering principle here:

> **Don't solve a problem before measuring that you actually have it.**

So I don't want us to immediately build a complicated dependency graph.

First we'll build the simplest version:

```
Repository
   ↓
Function-level chunks
   ↓
Embeddings
   ↓
Top-K retrieval
```

Then we'll deliberately create questions where it fails.

For example:

> "Explain how refunds are processed."

If the system retrieves only:

```
refund_payment()
```

and produces an incomplete answer, **we have evidence that context is insufficient.**

Then we can experiment with:

`Top-K = 3`

versus:

`Top-K = 5`

versus:

`Top-K + dependencies`

And measure which approach actually improves answer quality.

🎯 This is exactly the kind of thinking I want from you

You didn't say:

> "The correct chunk size is 500 tokens."

You said:

> **"There is a tradeoff, so perhaps function-level chunks + Top-K can reduce accidental information loss."**

That's **engineering reasoning**.

And eventually your interview explanation could be:

> "I initially considered file-level indexing, but it reduced retrieval precision because unrelated functions were embedded together. I moved toward function-level chunks and Top-K retrieval, but found that isolated functions sometimes lacked dependency context. That led me to investigate context preservation and dependency-aware retrieval."

That is a **much stronger interview story** than:

"I used LangChain to make a code RAG system."

One correction to your idea, though:

Top-K doesn't automatically solve context loss.

It *reduces the probability* of missing relevant information because we're retrieving multiple candidates.

But if the relevant dependency has a low similarity score, it may still never make it into Top-K.

That's the exact problem we'll investigate next. 🔥

So our next step should be:

Function-level chunking → Top-K → deliberately create a failure case → understand why retrieval misses dependencies → then design a better strategy.

You said:

got it why the answer is not precise what is the next lesson

ChatGPT said:

Exactly. You got the main reason: **retrieval precision vs context preservation**.

🚀 Next Lesson: Intelligent Code Chunking

This is the next major piece of GitContext:

```
Raw Repository
      ↓
   Parsing
      ↓
Intelligent Chunking
      ↓
Function/Class-level chunks + metadata
      ↓
   Embeddings
```

We already know **why** we don't want to blindly embed an entire file. Now we'll learn **how GitContext decides what a chunk should be**.

We'll cover:

1. **What exactly is a chunk?**
2. Why normal text chunking like "every 500 characters" is bad for code
3. **Function-level vs class-level vs file-level chunks**
4. How metadata helps:

   ```
   file: auth.py
   function: verify_password
   class: AuthService
   imports: bcrypt
   ```

5. **Context preservation** — how do we avoid losing dependencies?
6. What **AST (Abstract Syntax Tree)** is and why serious code RAG systems use it
7. Build our **first mini code chunker**
8. Break it intentionally with a dependency example
9. Fix it

And then we'll move to:

```
Chunking
   ↓
Embeddings
   ↓
Vector Database
   ↓
Hybrid Retrieval (Vector + BM25)
```

One important idea before we start

Your statement:

> "divide into functions and retrieve top-k"

is **directionally correct**, but GitContext needs to answer a harder question:

> **"What other code should come along with the function I retrieved so that the LLM actually understands it?"**

For example:

```
def process_payment(amount):
    if not validate_payment(amount):
        return False
```

```
    tax = calculate_tax(amount)
    return charge_card(amount + tax)
```

Suppose retrieval returns only `process_payment()` .

The LLM sees:

```
validate_payment(...)
calculate_tax(...)
charge_card(...)
```

but doesn't know what they do.

So the next lesson is essentially:

> **How do we create chunks that are small enough for precise retrieval, but rich enough to preserve code meaning?**

That's where **intelligent code chunking** begins.

You said:

lets gog

ChatGPT said:

Absolutely. 🔥 Let's start **Intelligent Code Chunking** from zero.

Lesson 3 — Intelligent Code Chunking

We already know:

```
Repository
   ↓
Files
   ↓
Chunks
   ↓
Embeddings
   ↓
Semantic Search
```

The question is:

What exactly should one chunk contain?

---

1. First: what is a chunk?

A **chunk** is simply a piece of source code that we treat as one retrievable unit.

Suppose we have:

```
# auth.py

def login_user(username, password):
    user = find_user(username)

    if verify_password(password, user.password_hash):
        return create_session(user)

    return None


def verify_password(password, password_hash):
    return bcrypt.checkpw(password, password_hash)


def logout_user(user):
    delete_session(user)
```

We could make the entire file one chunk:

```
Chunk 1 = entire auth.py
```

Or:

```
Chunk 1 = login_user()
Chunk 2 = verify_password()
Chunk 3 = logout_user()
```

That's **function-level chunking**.

---

2. Why not just split every 500 characters?

This is very common with normal document RAG.

For example:

```
characters 1–500 → chunk 1
characters 501–1000 → chunk 2
```

For natural language, this can sometimes work.

But code has **structure**.

Imagine splitting here:

```
def process_payment(amount):
    transaction = create_transaction(amount)

    if transaction.status == "pending":
        ...
```

and the next chunk starts:

```
        approve_transaction(transaction)

    return transaction
```

We just separated one function into two pieces.

That's terrible for code retrieval.

The embedding of chunk 2 doesn't really represent a complete concept.

---

3. Code has natural boundaries

Programming languages already tell us where meaningful units are.

For example:

```
File
 ├── imports
 ├── class
 │    ├── method
 │    ├── method
```

```
│    └── method
├── function
└── function
```

So instead of:

```
every 500 characters
```

we can use:

```
function
class
method
module
```

as natural boundaries.

This is the beginning of **structure-aware chunking**.

---

4. But now we hit a problem

Consider:

```
def process_payment(amount):
    if not validate_payment(amount):
        return False

    tax = calculate_tax(amount)

    return charge_card(amount + tax)
```

Suppose our chunks are:

```
Chunk A:
process_payment()

Chunk B:
validate_payment()

Chunk C:
calculate_tax()
```

```
Chunk D:
charge_card()
```

User asks:

> "How does payment processing validate the payment?"

Semantic search might retrieve:

```
Chunk A
Chunk B
```

Great.

But what if the user asks:

> "Why does process_payment return False?"

Maybe we retrieve:

```
Chunk A
```

That's enough.

But now ask:

> "What tax calculation is used during payment processing?"

We might retrieve:

```
Chunk A
Chunk C
```

Also good.

---

5. The real problem: dependencies

Now imagine this:

```
# payments.py
```

```
def process_payment(amount):
    if not validate_payment(amount):
        return False

    tax = calculate_tax(amount)

    return charge_card(amount + tax)
```

And somewhere else:

```
# validation.py

def validate_payment(amount):
    return amount > 0 and amount < MAX_PAYMENT
```

Now the retrieved chunk might be:

```
payments.py
process_payment()
```

But the meaning of:

```
validate_payment(amount)
```

is missing.

The LLM doesn't know:

```
What makes a payment valid?
```

This gives us an important distinction.

Retrieval precision

Did we retrieve the **right code**?

Context completeness

Did we retrieve **enough related code to understand it**?

These are not the same thing.

---

6. This gives us our first GitContext design problem

We want:

```
Small chunks
      ↓
High retrieval precision
```

but also:

```
Related context
      ↓
Better understanding
```

So GitContext eventually needs something like:

```
User Query
    ↓
Retrieve process_payment()
    ↓
Find dependencies
    ↓
Retrieve validate_payment()
    ↓
Retrieve calculate_tax()
    ↓
Assemble useful context
    ↓
LLM
```

That's much more powerful than simply:

```
query → top 5 chunks → LLM
```

But **we are NOT implementing dependency-aware retrieval yet**.

First we need to understand how to create good chunks.

---

7. What should a code chunk contain?

A naïve chunk could be:

```
def verify_password(password, password_hash):
    return bcrypt.checkpw(password, password_hash)
```

But GitContext can store additional metadata:

```
Code:
def verify_password(password, password_hash):
    return bcrypt.checkpw(password, password_hash)

Metadata:
    file = auth.py
    function = verify_password
    language = python
    line_start = 12
    line_end = 13
```

Now our chunk isn't just:

```
CODE
```

It is:

```
CODE + INFORMATION ABOUT THE CODE
```

This metadata becomes **extremely useful later**.

For example, the answer could say:

> `verify_password()` in `auth.py` uses bcrypt to compare the supplied password against the stored password hash.

And GitContext can provide a citation such as:

```
auth.py:12-13
```

---

8. Our first chunk representation

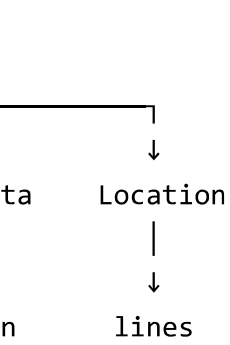
Before using a vector database, let's represent chunks using ordinary Python dictionaries.

For example:

```
chunk = {
    "content": """
def verify_password(password, password_hash):
    return bcrypt.checkpw(password, password_hash)
""",

    "file": "auth.py",
    "function": "verify_password",
    "language": "python",
    "start_line": 12,
    "end_line": 13
}
```

Think of this as:



Later:

```
chunk
 ↓
embedding
 ↓
vector DB
```

The embedding represents the **semantic meaning**.

The metadata helps us **filter, explain, cite and navigate**.

---

9. Why metadata matters for retrieval

Suppose the user asks:

> "Show me authentication code."

We might search semantically.

But suppose they ask:

> "What functions are inside `auth.py` ?"

That's not really a semantic similarity problem.

We already know:

```
file = auth.py
```

So metadata can answer/filter this directly.

Similarly:

> "Show me Python functions related to payment."

Could combine:

```
semantic similarity
+
language = python
+
file/path metadata
```

This is why serious RAG systems don't just store:

```
vector
```

They store something closer to:

```
vector
+
content
+
metadata
+
relationships
```

---

10. But how do we actually detect functions?

Here's where things get interesting.

We **could** do something terrible like:

```
text.split("\n\n")
```

But that's not reliable.

We need the computer to understand:

```
"This is a function."

"This is a class."

"This is a method."

"This function starts at line 15."

"This function ends at line 27."
```

That brings us to:

🌳 AST — Abstract Syntax Tree

Don't worry. It sounds much harder than it is.

Consider:

```
def add(a, b):
    return a + b
```

Python doesn't fundamentally see this as a bunch of characters.

It can parse it into a tree conceptually like:

```
FunctionDef
│
├── name: add
│
├── arguments
│   ├── a
│   └── b
│
```

```
└── body
    └── Return
        └── a + b
```

That's an **Abstract Syntax Tree**.

The tree represents the **structure of the program**.

---

11. Why AST is perfect for GitContext

Instead of asking:

> "Where are the next 500 characters?"

we can ask:

> "Give me every function definition."

Then Python's parser can tell us:

```
login_user
verify_password
logout_user
```

We can turn each into a chunk.

So:

```
Raw code
   ↓
AST parser
   ↓
Functions/classes
   ↓
Chunks
   ↓
Metadata
   ↓
Embeddings
```

This is much more intelligent than arbitrary text splitting.

12. Tiny challenge for you 🧠

Don't code yet.

Suppose we have:

```
class PaymentService:

    def validate(self, amount):
        return amount > 0

    def calculate_tax(self, amount):
        return amount * 0.18


def send_email(user, message):
    ...
```

If we're building GitContext, how many **natural code units** would you initially consider?

Would you choose:

A.

```
1 chunk = entire file
```

B.

```
3 chunks:
PaymentService
validate()
calculate_tax()
send_email()
```

C.

```
2 chunks:
PaymentService
send_email()
```

And more importantly:

Why?

Think about the retrieval precision **and** context preservation tradeoff we just learned.

You said:

next

ChatGPT said:

Good. Let's keep moving. You don't need to answer every mini-question unless something is unclear.

Lesson 3.2 — Actually Building the Chunker

We now know **what** intelligent chunking means. Let's actually build a tiny version.

Our goal:

```
Python file
    ↓
Find functions
    ↓
Extract each function
    ↓
Store code + metadata
```

---

1. Our input

Create a file called `auth.py`:

```
import bcrypt

MAX_LOGIN_ATTEMPTS = 5


def verify_password(password, password_hash):
    return bcrypt.checkpw(password, password_hash)


def login_user(username, password):
    user = find_user(username)

    if verify_password(password, user.password_hash):
        return create_session(user)
```

```
    return None


def logout_user(user):
    delete_session(user)
```

We want the computer to discover:

```
Chunk 1 → verify_password()
Chunk 2 → login_user()
Chunk 3 → logout_user()
```

---

2. The naïve way

We could manually do something like:

```
functions = [
    "verify_password",
    "login_user",
    "logout_user"
]
```

But obviously this doesn't scale.

A real repository could have:

```
500 files
20,000 functions
```

We need the computer to understand the code structure automatically.

That's where Python's `ast` module comes in.

---

3. AST parser

Python already has an AST parser built into the standard library.

So:

```
import ast
```

Then:

```
tree = ast.parse(source_code)
```

Conceptually:

```
source_code
     ↓
  ast.parse()
     ↓
Abstract Syntax Tree
```

We can then inspect the tree.

---

4. Finding functions

The important AST node for a normal Python function is:

```
ast.FunctionDef
```

So we can walk through the tree:

```
for node in ast.walk(tree):
    if isinstance(node, ast.FunctionDef):
        print(node.name)
```

What happens?

Suppose:

```
def login_user(...):
    ...


def logout_user(...):
    ...
```

The AST contains corresponding `FunctionDef` nodes.

So:

```
isinstance(node, ast.FunctionDef)
```

basically asks:

"Is this particular piece of the syntax tree a function definition?"

If yes:

`node.name`

gives:

`login_user`

or:

`logout_user`

---

5. But we need the actual source code too

Knowing:

`function = login_user`

isn't enough.

We need:

```
def login_user(username, password):
    user = find_user(username)

    if verify_password(password, user.password_hash):
        return create_session(user)

    return None
```

Modern Python AST nodes contain source location information.

For example:

`node.lineno`

gives the starting line.

And:

`node.end_lineno`

gives the ending line.

So we can know:

```
login_user
start = 9
end = 16
```

Then we can extract those lines from the original source.

---

6. Let's build it

Create:

`chunker.py`

Start with:

```
import ast
```

Then read the file:

```
with open("auth.py", "r") as file:
    source = file.read()
```

Now:

```
tree = ast.parse(source)
```

Now we have:

```
source
  ↓
tree
```

---

7. Find all functions

```
for node in ast.walk(tree):

    if isinstance(node, ast.FunctionDef):

        print("Function:", node.name)
```

```
        print("Start:", node.lineno)
        print("End:", node.end_lineno)
```

For our file, we'd get approximately:

```
Function: verify_password
Start: 6
End: 7

Function: login_user
Start: 10
End: 16

Function: logout_user
Start: 19
End: 20
```

Notice something important.

We haven't used:

```
LangChain
LlamaIndex
Qdrant
Pinecone
OpenAI
```

Nothing.

We're understanding the **actual underlying mechanism** first.

That's exactly what we want for GitContext.

---

8. Now extract the source

We have the original source:

`source`

and a function node:

`node`

We can split the source into lines:

```
lines = source.splitlines()
```

Suppose:

```
line 1 → import bcrypt
line 2 → blank
line 3 → MAX_LOGIN_ATTEMPTS = 5
...
```

Python's AST line numbers start at **1**.

Python lists start at **0**.

So:

```
start = node.lineno - 1
end = node.end_lineno
```

Then:

```
function_code = "\n".join(lines[start:end])
```

This gives us exactly that function.

---

9. Now we can create our first real chunk

```
chunk = {
    "content": function_code,
    "file": "auth.py",
    "function": node.name,
    "start_line": node.lineno,
    "end_line": node.end_lineno
}
```

Now imagine:

`chunk`

contains:

```
content:
def login_user(username, password):
    user = find_user(username)
```

```
    if verify_password(password, user.password_hash):
        return create_session(user)

    return None

file:
auth.py

function:
login_user

start_line:
10

end_line:
16
```

🎯 This is a proper code chunk.

---

10. Why this is much better than arbitrary splitting

Compare:

Bad approach

```
characters 1–500
characters 501–1000
```

versus:

Our approach

```
AST
 ↓
Function boundary
 ↓
Complete function
 ↓
Metadata
```

Our chunk has a semantic identity:

> `" login_user() in auth.py "`

rather than:

> "characters 1001–1500"

That's a huge improvement.

---

11. BUT... we just discovered another problem

Look at our chunk:

```
def login_user(username, password):
    user = find_user(username)

    if verify_password(password, user.password_hash):
        return create_session(user)

    return None
```

It uses:

```
find_user()
verify_password()
create_session()
```

But our chunk doesn't contain their implementations.

So our chunker solved:

> **"How do I make meaningful chunks?"**

But it did **not** solve:

> **"How do I preserve relationships between chunks?"**

And this is where GitContext becomes significantly more interesting.

---

12. The next concept: Code relationships

Think of the repository as a graph:

```
login_user()
   │
   ├──── calls ────→ find_user()
   │
   ├──── calls ────→ verify_password()
   │
   └──── calls ────→ create_session()
```

Instead of treating chunks as independent pieces:

```
Chunk A
Chunk B
Chunk C
Chunk D
```

we can eventually understand:

```
        login_user
        /    |    \
       ↓     ↓     ↓
find_user  verify  create_session
```

Now retrieval can potentially work like:

```
User query
    ↓
retrieve login_user()
    ↓
inspect relationships
    ↓
retrieve relevant dependencies
    ↓
assemble context
```

This is the beginning of **dependency-aware retrieval**.

---

13. But don't jump there yet

There is an important engineering principle here:

> **Don't solve a problem before you have measured it.**

Our current system is:

```
AST chunking
      ↓
Function chunks
      ↓
Embeddings
      ↓
Top-K retrieval
```

Before making it complicated, we'll test it.

We'll intentionally ask questions where function-level retrieval works:

> "How is the password verified?"

And questions where it can fail:

> "Why does login fail when the password is incorrect?"

Then we'll inspect:

```
What did retrieval return?
What information was missing?
```

That gives us an actual failure case.

Then—and only then—we improve the system.

Next lesson

Lesson 3.3 — Dependency-aware retrieval

We'll build a tiny repository where:

```
login_user()
     ↓
verify_password()
     ↓
bcrypt
```

and see why **Top-K semantic retrieval alone can fail**, then learn how GitContext can use code relationships to recover the missing context.

That's where our simple RAG starts becoming an actual **code intelligence system**.

You said:

yes go ahead

ChatGPT said:

Let's go. This is one of the **most important ideas in GitContext**, because this is where we move from "semantic search over code" toward actual **codebase understanding**.

Lesson 3.3 — Dependency-Aware Retrieval

We currently have:

```
Query
  ↓
Embedding
  ↓
Top-K similar chunks
  ↓
LLM
```

This works surprisingly well for simple questions.

But code has **relationships**.

And those relationships can make normal Top-K retrieval fail.

---

1. Let's create a tiny repository

Imagine:

```
project/
├── auth.py
├── users.py
└── session.py
```

auth.py

```
def login_user(username, password):
    user = find_user(username)

    if verify_password(password, user.password_hash):
        return create_session(user)

    return None


def verify_password(password, password_hash):
    return bcrypt.checkpw(password, password_hash)
```

users.py

```
def find_user(username):
    return database.get_user(username)
```

session.py

```
def create_session(user):
    return generate_jwt(user.id)
```

Now look at the relationship:

```
                login_user()
               /     |      \
              ↓      ↓       ↓
      find_user()  verify()  create_session()
         users.py   auth.py      session.py
```

This is crucial.

The repository isn't just a collection of documents.

It's a **network of connected code**.

---

2. What happens with normal semantic retrieval?

User asks:

> **"How does login authenticate a user?"**

We embed the query.

Then compare it against:

```
login_user()
verify_password()
find_user()
create_session()
```

Suppose our similarity scores are:

```
login_user()       0.86
verify_password()  0.82
create_session()   0.61
find_user()        0.55
```

If:

```
K = 2
```

we retrieve:

```
1. login_user()
2. verify_password()
```

That's actually pretty good.

The LLM can probably answer.

---

3. But change the question

User asks:

> **"Where does the application get the user's information before checking the password?"**

Now imagine the scores are:

```
login_user()       0.73
find_user()        0.71
verify_password()  0.54
create_session()   0.20
```

Top-2:

```
login_user()
find_user()
```

Again, good.

---

4. Now the interesting failure

Suppose the user asks:

> **"What algorithm is used to create the authentication session after successful login?"**

Potential scores:

```
login_user()       0.69
create_session()   0.68
verify_password()  0.48
find_user()        0.30
```

Top-2 gives:

```
login_user()
create_session()
```

Perfect.

So where is the problem?

The problem becomes obvious when the required information is **distributed across multiple related functions**, but their semantic similarity isn't high enough.

For example:

```
def process_payment(amount):
    transaction = create_transaction(amount)
```

```
    if validate_transaction(transaction):
        return execute_payment(transaction)

    return reject(transaction)
```

And:

```
def validate_transaction(transaction):
    return (
        transaction.user.is_verified
        and transaction.amount <= DAILY_LIMIT
        and transaction.status == "pending"
    )
```

The question:

> **"Why can a payment be rejected?"**

might retrieve:

```
process_payment()
reject()
```

But not:

```
validate_transaction()
```

And therefore the LLM sees:

```
if validate_transaction(transaction):
    ...
else:
    return reject(transaction)
```

but doesn't know **what validation means**.

---

5. Here's the fundamental problem

Semantic retrieval asks:

> **"Which chunks look most similar to my question?"**

But code understanding sometimes requires:

> **"Which chunks are related to the chunks I already retrieved?"**

Those are different questions.

That's the key insight.

---

6. Think of it like Google Maps

Imagine you want to reach:

`College`

Semantic retrieval tells you:

> "Here are roads that look relevant to the destination."

But sometimes you need:

```
Road A
   ↓
Road B
   ↓
Road C
   ↓
College
```

The roads aren't individually similar to the destination.

They're **connected**.

Code works similarly.

```
process_payment()
       ↓
validate_transaction()
```

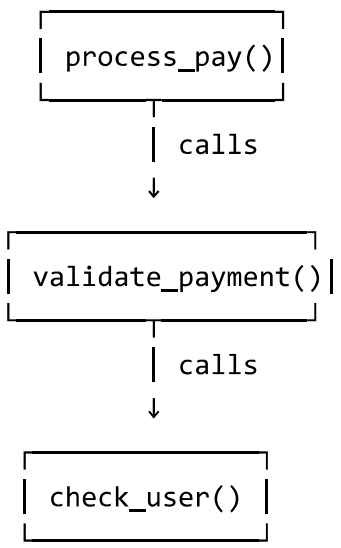
```
      ↓
check_user_status()
```

Even if `check_user_status()` isn't semantically similar to the user's question, it may be **structurally essential**.

---

7. So let's introduce a graph

We can represent our codebase as:



Each function is a **node**.

A relationship is an **edge**.

For example:

`A ──calls──→ B`

This gives us a **code graph**.

---

8. How do we create this graph?

Remember our AST?

We already used AST to find:

`FunctionDef`

But AST can tell us much more.

Suppose:

```
def login_user(username, password):
    user = find_user(username)

    if verify_password(password, user.password_hash):
        return create_session(user)
```

The AST can identify function calls:

```
find_user()
verify_password()
create_session()
```

So we can record:

```
{
    "function": "login_user",
    "calls": [
        "find_user",
        "verify_password",
        "create_session"
    ]
}
```

Now we have a relationship.

---

9. Our chunk becomes more powerful

Previously:

```
{
    "content": "...",
    "file": "auth.py",
    "function": "login_user"
}
```

Now:

```
{
    "content": "...",
```

```
    "file": "auth.py",
    "function": "login_user",
    "calls": [
        "find_user",
        "verify_password",
        "create_session"
    ]
}
```

Notice what happened.

The embedding still represents the **semantic content**.

But metadata now tells us about **relationships**.

---

10. Retrieval can now happen in two stages

Instead of:

```
Query
 ↓
Top-K
 ↓
LLM
```

we can eventually do:

```
Query
 ↓
Semantic Retrieval
 ↓
Top-K chunks
 ↓
Inspect relationships
 ↓
Retrieve relevant dependencies
 ↓
Context Assembly
 ↓
LLM
```

For example:

```
Query:
"Why does login fail?"

          ↓

Semantic retrieval

          ↓

login_user()
verify_password()

          ↓

Look at relationships

login_user()
 ├── find_user()
 ├── verify_password()
 └── create_session()

          ↓

Fetch useful related chunks

          ↓

LLM
```

This is **dependency-aware retrieval**.

---

11. But here's an important warning ⚠️

You might now think:

> "Easy! Just retrieve every dependency."

No.

Imagine:

```
login_user()
   ↓
A()
   ↓
B()
   ↓
C()
   ↓
D()
   ↓
E()
   ↓
F()
```

If we retrieve everything, our context explodes.

And then we're back to our original problem:

```
Too much context
       ↓
Noise
       ↓
Higher token cost
       ↓
Potentially worse answers
```

So now we have another tradeoff:

Too little context

```
Precise
but incomplete
```

Too much context

```
Complete
but noisy/expensive
```

We need the **right amount of context**.

---

12. This is why Top-K is not the final answer

Earlier we said:

> "Retrieve Top-K."

That's a good starting point.

But in a serious codebase intelligence system, we may eventually have:

```
Top-K semantic chunks
        +
dependency expansion
        +
metadata filtering
        +
reranking
        +
context budget
```

That is much closer to production-grade retrieval.

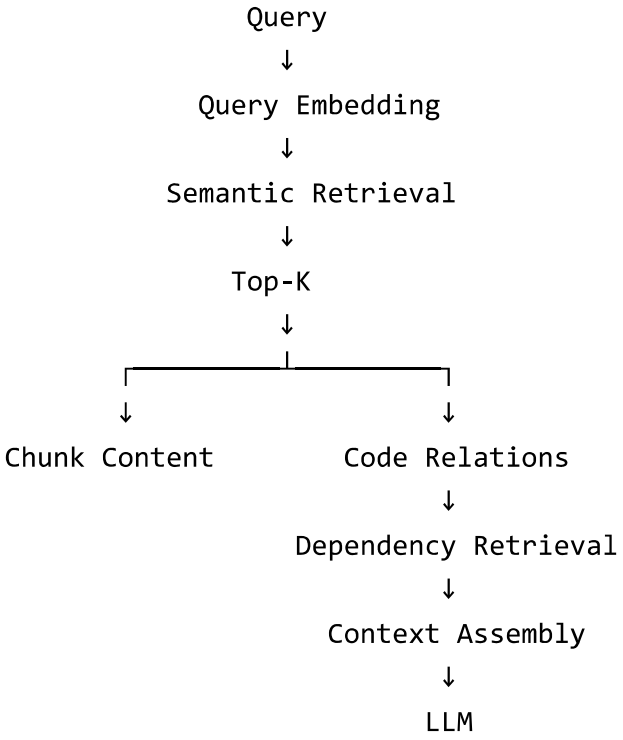
---

13. Our GitContext retrieval architecture is evolving

Initially:

```
Query
  ↓
Embedding
  ↓
Vector Search
  ↓
Top-K
  ↓
LLM
```

Now:



And we're still not finished.

---

14. One more important concept: not every relationship is a dependency

Code can have different relationships:

```
calls
imports
inherits
implements
references
defines
tests
```

For example:

```
PaymentService
      │
      ├── calls → validate_payment()
      │
      ├── imports → PaymentRepository
```

```
    │
    └── inherits → BaseService
```

A sophisticated GitContext can eventually understand these different relationship types.

---

15. What we're going to do next

Before adding vector databases or fancy frameworks, we're going to build this ourselves in a tiny experiment:

```
repo/
├── auth.py
├── users.py
└── session.py
```

We'll use Python's AST to extract:

```
Function
   ↓
Calls
   ↓
Dependencies
```

So we'll actually produce something like:

```
{
    "name": "login_user",
    "file": "auth.py",
    "calls": [
        "find_user",
        "verify_password",
        "create_session"
    ]
}
```

Then we'll combine that with our existing embeddings.

That will give you a very important understanding:

> **Embeddings tell us what code is semantically relevant.**
> **The code graph tells us what code is structurally related.**

And **GitContext needs both**.

Next → Lesson 3.4: Building the Code Dependency Graph with AST

We'll write the code **line by line**, and I'll explain exactly how AST discovers `find_user()`, `verify_password()`, etc.

You said:

next

ChatGPT said:

Lesson 3.4 — Building the Code Dependency Graph

Now we're going to **actually build** the relationship information we just discussed.

Our goal is simple:

```
login_user()
    ├── calls → find_user()
    ├── calls → verify_password()
    └── calls → create_session()
```

We already know AST can find functions. Now we'll make it find **function calls inside functions**.

---

1. Our tiny codebase

Let's use one file first so we don't introduce too many problems at once.

```
def find_user(username):
    return database.get_user(username)


def verify_password(password, password_hash):
    return bcrypt.checkpw(password, password_hash)


def create_session(user):
    return generate_jwt(user.id)
```

```
def login_user(username, password):
    user = find_user(username)

    if verify_password(password, user.password_hash):
        return create_session(user)

    return None
```

We want the output:

```
find_user       → []
verify_password → []
create_session  → []
login_user      → [find_user, verify_password, create_session]
```

---

2. First question: how does AST see this?

Take:

```
user = find_user(username)
```

AST doesn't simply see the string:

```
"find_user"
```

It sees a **Call node**.

Conceptually:

```
Call
├── function → find_user
└── arguments
    └── username
```

So Python gives us:

```
ast.Call
```

This is analogous to:

```
ast.FunctionDef
```

which represented a function definition.

Therefore:

```
FunctionDef → "A function was defined here"

Call        → "A function/method was called here"
```

---

3. Let's inspect calls

Our first experiment:

```
import ast

source = """
def login_user(username, password):
    user = find_user(username)

    if verify_password(password, user.password_hash):
        return create_session(user)
"""

tree = ast.parse(source)

for node in ast.walk(tree):
    if isinstance(node, ast.Call):
        print(node)
```

What are we doing?

```
source
  ↓
ast.parse()
  ↓
tree
  ↓
walk through every node
  ↓
find ast.Call
```

We should find three calls:

```
find_user(...)
verify_password(...)
create_session(...)
```

---

4. But we don't want the whole Call node

If we print:

`print(node)`

we get an AST representation, not simply:

`find_user`

We need the name of the function being called.

For:

`find_user(username)`

the AST roughly looks like:

```
Call
│
├── func
│     └── Name
│           └── id = "find_user"
│
└── args
      └── username
```

So:

`node.func`

gets the function being called.

And:

`node.func.id`

gets:

`find_user`

---

5. Our first useful code

```
for node in ast.walk(tree):

    if isinstance(node, ast.Call):

        if isinstance(node.func, ast.Name):
            print(node.func.id)
```

Let's break this down carefully.

First:

`ast.walk(tree)`

means:

> Visit every node in the AST.

Then:

`isinstance(node, ast.Call)`

means:

> Is this node a function/method call?

If not, ignore it.

Then:

`isinstance(node.func, ast.Name)`

Why another check?

Because not every function call has the simple form:

`function()`

You can have:

`obj.method()`

We'll deal with that later.

For now we're handling simple calls like:

`find_user()`

Finally:

`node.func.id`

gives:

`find_user`

---

6. But there's a problem

Our current code finds:

```
find_user
verify_password
create_session
```

But it doesn't tell us:

> **Which function called them?**

We want:

```
login_user
    ↓
find_user
verify_password
create_session
```

Not merely:

```
find_user
verify_password
create_session
```

So we need to associate each call with its surrounding function.

---

7. Remember our FunctionDef nodes?

We can visit each function individually.

Conceptually:

```
FunctionDef: login_user
      ↓
look inside it
      ↓
find Call nodes
```

That is much better.

Let's do:

```
for node in ast.walk(tree):

    if isinstance(node, ast.FunctionDef):

        print("FUNCTION:", node.name)

        for child in ast.walk(node):

            if isinstance(child, ast.Call):

                if isinstance(child.func, ast.Name):
                    print("  CALLS:", child.func.id)
```

Now the structure becomes:

```
FUNCTION: find_user

FUNCTION: verify_password

FUNCTION: create_session
```

```
FUNCTION: login_user
  CALLS: find_user
  CALLS: verify_password
  CALLS: create_session
```

🎯 We have our first dependency information.

---

8. Turn it into data

Instead of printing, let's store it.

```
dependencies = {}

for node in ast.walk(tree):

    if isinstance(node, ast.FunctionDef):

        calls = []

        for child in ast.walk(node):

            if isinstance(child, ast.Call):

                if isinstance(child.func, ast.Name):
                    calls.append(child.func.id)

        dependencies[node.name] = calls
```

Now:

```
print(dependencies)
```

would conceptually produce:

```
{
    'find_user': [],
    'verify_password': [],
    'create_session': [],
    'login_user': [
        'find_user',
        'verify_password',
        'create_session'
```

```
    ]
}
```

That is our first **code dependency graph representation**.

---

9. Why call this a graph?

Because we can represent:

```
login_user → find_user
login_user → verify_password
login_user → create_session
```

as edges.

Mathematically:

```
G = (V, E)
```

where:

- `V` = vertices/nodes = functions
- `E` = edges = relationships between functions

Our nodes:

```
V =
{
find_user,
verify_password,
create_session,
login_user
}
```

Our edges:

```
E =
{
(login_user, find_user),
(login_user, verify_password),
(login_user, create_session)
}
```

So our repository has become a graph.

10. Now combine this with our chunks

Previously our chunk looked like:

```
{
    "content": "...",
    "file": "auth.py",
    "function": "login_user",
    "start_line": 10,
    "end_line": 16
}
```

Now we can add:

```
"calls": [
    "find_user",
    "verify_password",
    "create_session"
]
```

So:

```
{
    "content": "...",
    "file": "auth.py",
    "function": "login_user",
    "start_line": 10,
    "end_line": 16,
    "calls": [
        "find_user",
        "verify_password",
        "create_session"
    ]
}
```

Now the chunk knows both:

```
WHAT IT SAYS
+
WHAT IT CONNECTS TO
```

That's a major upgrade.

11. Now let's make retrieval smarter

Suppose the query is:

> "How does login verify a user's password?"

Semantic search returns:

```
login_user()
verify_password()
```

Good.

Now GitContext can inspect:

```
login_user()
calls:
  find_user
  verify_password
  create_session
```

It knows:

```
verify_password
```

is directly related.

So it can ensure that chunk is included.

---

12. But there is a deeper problem

Look at:

```
def login_user(username, password):
    user = find_user(username)

    if verify_password(password, user.password_hash):
        return create_session(user)
```

What if `verify_password()` itself calls something?

```
def verify_password(password, password_hash):
    return check_hash(password, password_hash)
```

And:

```
def check_hash(password, password_hash):
    return bcrypt.checkpw(password, password_hash)
```

Now:

```
login_user
    ↓
verify_password
    ↓
check_hash
    ↓
bcrypt
```

Our dependency graph isn't just one level deep.

It's a **chain**.

---

13. How far should we follow dependencies?

This becomes another engineering decision.

Depth = 1

```
login_user
    ↓
verify_password
```

Small context.

Depth = 2

```
login_user
    ↓
verify_password
    ↓
check_hash
```

More context.

Depth = 10

Potentially:

```
login_user
 ↓
A
 ↓
B
 ↓
C
 ↓
D
 ↓
E
...
```

Huge context.

So eventually we'll need something like:

```
maximum dependency depth
```

or a relevance-based expansion strategy.

Again:

> **More context ≠ automatically better.**

---

14. One more important problem: methods

Our code so far has:

```
find_user()
```

But real repositories contain:

```
database.get_user(username)
```

The AST sees:

```
Call
└── Attribute
```

```
    ├── value → database
    └── attr → get_user
```

instead of:

```
Call
└── Name
    └── id → get_user
```

So our current code:

```
isinstance(child.func, ast.Name)
```

would **miss**:

```
database.get_user()
```

This is an example of a deliberate failure.

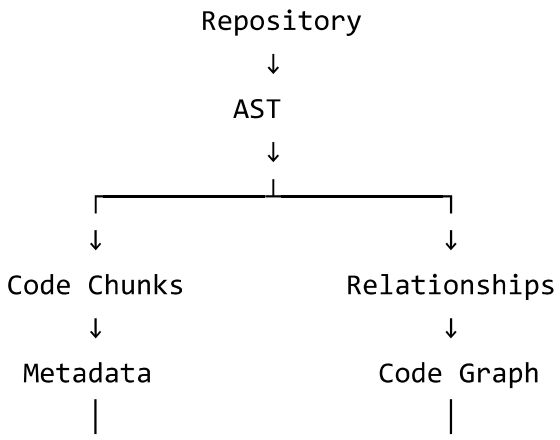
And this is exactly how we're going to build GitContext properly:

```
Build simple version
       ↓
Break it
       ↓
Understand failure
       ↓
Improve it
```

---

15. Our current architecture

We now have:



```
└──────────┬──────────┘
           ↓
      Retrieval
```

This is getting much closer to a real **code intelligence engine**.

🔥 The next lesson is important

We've now learned:

- function-level chunking
- metadata
- AST
- function calls
- dependency graphs
- dependency-aware retrieval
- context expansion

But we're still using **only semantic/vector search**.

Real GitContext should handle a query like:

```
Where is PAY_4037 handled?
```

Very differently from:

```
How does the payment authorization flow work?
```

The first is an **exact identifier lookup**.

The second is a **semantic understanding** problem.

That leads us to our next major component:

Lesson 4 — Hybrid Retrieval: Vector Search + BM25

We'll learn **why embeddings alone are not enough for code**, what BM25 actually does internally, and then combine:

```
Semantic Search
      +
```

```
Keyword Search
      ↓
Hybrid Retrieval
```

This is one of the most important upgrades to our GitContext architecture.

You said:

lets goo

ChatGPT said:

🚀 Lesson 4 — Hybrid Retrieval: Vector Search + BM25

This is a **big milestone**.

Until now, our retrieval system basically thinks:

> "Find code whose *meaning* is similar to the question."

That is powerful, but code has a special problem:

> **Exact words matter enormously.**

Let's see why.

---

1. The problem with embeddings alone

Suppose your repository contains:

```
PAY_4037 = "Payment authorization failed"
```

and the user asks:

> **"Where is PAY_4037 handled?"**

Our embedding model converts the query into a vector.

It might retrieve:

```
payment authorization logic
payment failure handler
transaction error handling
```

because those are semantically related.

But the user specifically wants:

```
PAY_4037
```

The exact identifier is what matters.

Imagine another file:

```
if error_code == "PAY_4037":
    retry_payment()
```

That exact match is incredibly valuable.

---

2. Semantic search and keyword search are good at different things

Think of two specialists.

🧠 Semantic search

Good at:

> "How does the system authenticate users?"

It can find:

```
verify_password()
login_user()
authenticate()
```

even if the exact word "authenticate" isn't present.

🔎 Keyword search

Good at:

> "Where is `PAY_4037` ?"

It looks for:

`PAY_4037`

exactly.

So:

```
Semantic Search
→ meaning

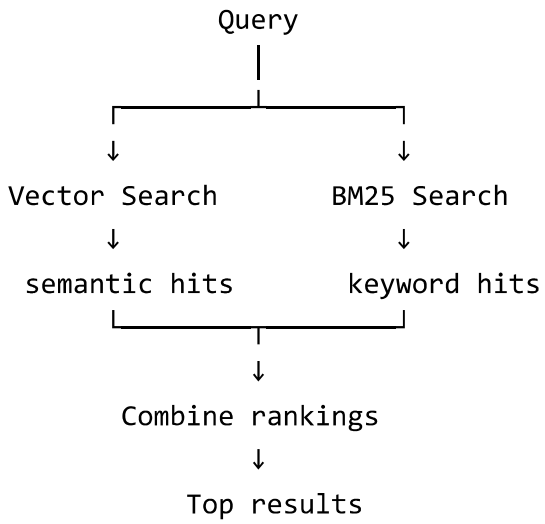
Keyword Search
→ exact terms
```

Neither is universally better.

---

3. This is why GitContext needs both

Our architecture becomes:



```
                    ↓
                   LLM
```

This is called:

Hybrid Retrieval

And one of the classic keyword retrieval algorithms we'll use is **BM25**.

---

4. Before BM25: understand the problem it solves

Imagine our repository contains:

```
Document A:
def login_user():
    verify_password()
    create_session()

Document B:
def verify_password():
    bcrypt.checkpw()

Document C:
def send_password_reset_email():
    ...
```

Query:

```
password
```

All three documents contain "password" somewhere.

But they aren't equally useful.

We need a way to score:

> How relevant is this document to this query?

That's what BM25 does.

5. BM25 intuition

BM25 basically asks:

Question 1

Does the document contain the query term?

If no:

```
score ≈ 0
```

If yes:

```
score ↑
```

Question 2

How rare is that word across the repository?

Suppose:

```
"function" → appears in 10,000 chunks
"PAY_4037" → appears in 2 chunks
```

Finding:

```
PAY_4037
```

is much more informative.

So rare terms get more importance.

This is called **Inverse Document Frequency (IDF)**.

Question 3

How often does the term occur in this document?

If:

```
Document A:
password appears 1 time
```

versus:

```
Document B:
password appears 8 times
```

Document B may be more relevant.

But BM25 doesn't blindly say:

> 8 occurrences = 8× more relevant.

It uses **term-frequency saturation**.

We'll get to that.

---

6. The BM25 formula

Don't panic. 😄

The standard BM25 score is:

$$BM25(D,Q) = \sum_{t \in Q} IDF(t) \cdot \frac{f(t,D)(k_1+1)}{f(t,D) + k_1(1 - b + b\frac{|D|}{avgdl})}$$

Let's decode it.

Q

The query.

Example:

```
PAY_4037 payment failure
```

D

One document/chunk.

For example:

`payment_handler.py`

`t`

One term from the query.

For example:

`PAY_4037`

`f(t,D)`

How many times term `t` occurs in document `D`.

Example:

`PAY_4037 appears 3 times`

Then:

`f(t,D) = 3`

`IDF(t)`

How rare the term is across the collection.

This is extremely important.

If:

`"payment" → appears everywhere`

then its IDF is relatively low.

If:

`"PAY_4037" → appears in only 2 chunks`

its IDF is high.

So BM25 says:

Rare words are stronger signals of relevance.

---

7. Why does document length matter?

Imagine:

Chunk A

```
PAY_4037
```

Chunk B

```
1000 lines of unrelated code...
...
PAY_4037
...
```

Both contain:

```
PAY_4037
```

But Chunk A is much more focused.

BM25 accounts for document length.

That's what:

```
|D| / avgdl
```

is doing.

Where:

- `|D|` = length of current document
- `avgdl` = average document length

---

8. What are k_1 and b?

You don't need to memorize the formula yet.

But understand their purpose.

k_1

Controls how quickly term frequency saturates.

For example:

`PAY_4037 appears once`

is useful.

But:

`PAY_4037 appears 100 times`

shouldn't make the document 100× more relevant.

So BM25 has diminishing returns.

---

b

Controls how strongly document length normalization affects the score.

Common values are around:

$k_1 \approx 1.2–2.0$
$b \approx 0.75$

You can tune them later.

---

9. Let's compare the two retrieval methods

Suppose our query is:

```
How does password authentication work?
```

Vector search might produce:

1. `verify_password()`
2. `login_user()`

```
3. authenticate_request()
4. session_manager()
```

Excellent semantic understanding.

But perhaps the query:

> `PAY_4037`

produces:

```
1. payment_failure_handler()
2. payment_service()
3. retry_transaction()
```

Maybe the exact constant wasn't ranked first.

BM25 might produce:

```
1. error_codes.py → PAY_4037
2. payment_handler.py → PAY_4037
3. retry.py → PAY_4037
```

Perfect for exact identifiers.

But if the query is:

> "How does the application verify a user's password?"

BM25 may struggle if the code uses:

`bcrypt.checkpw()`

without the word:

`verify`

Semantic search can connect those concepts.

---

10. This gives us a beautiful division of labor

Retrieval	Strong at
Vector	Meaning / semantic similarity
BM25	Exact words / identifiers
Vector	Natural-language questions
BM25	Error codes
Vector	Conceptual relationships
BM25	Function/class names
Vector	Similar phrasing
BM25	Rare technical terms

And codebases contain **both types of information**.

---

11. Now comes the interesting part: combining them

Suppose vector search gives:

```
login_user()       → 0.91
verify_password()  → 0.88
auth_middleware()  → 0.75
```

BM25 gives:

```
verify_password()  → 12.4
login_user()       → 8.7
auth_middleware()  → 2.1
```

These scores are **not directly comparable**.

Notice:

```
0.91
```

versus:

```
12.4
```

We can't simply do:

```
0.91 + 12.4
```

That would be meaningless.

This leads us to **rank fusion**.

---

12. The simplest approach: Reciprocal Rank Fusion

Don't worry about the name.

The idea is simple.

Suppose:

Vector ranking

```
Rank 1 → login_user
Rank 2 → verify_password
Rank 3 → create_session
```

BM25 ranking

```
Rank 1 → verify_password
Rank 2 → login_user
Rank 3 → payment_handler
```

We don't care about the raw scores.

We care about:

> **Where did each result rank?**

RRF gives each result a score based on its rank:

$$RRF(d) = \sum_{r} \frac{1}{k + rank_r(d)}$$

Usually `k` is some constant such as 60.

The important idea:

```
Rank 1 → high contribution
Rank 2 → slightly lower
Rank 3 → lower
...
```

If a chunk appears near the top in **both** retrieval systems, it gets a strong combined ranking.

---

13. Example

Vector:

```
login_user       Rank 1
verify_password  Rank 2
create_session   Rank 3
```

BM25:

```
verify_password  Rank 1
login_user       Rank 2
PAY_4037         Rank 3
```

RRF notices:

```
login_user
→ high in vector
→ high in BM25

verify_password
→ high in vector
→ #1 in BM25
```

So those two become strong candidates.

This is a very nice property:

A result supported by multiple retrieval signals gets promoted.

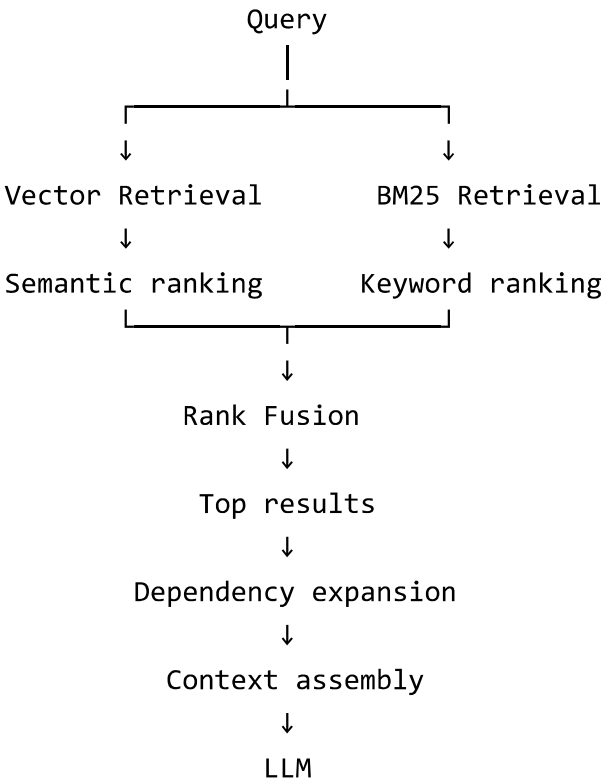
---

14. Our GitContext architecture is now getting serious

We started with:

```
Query
 ↓
Embedding
 ↓
Vector Search
 ↓
Top-K
```

Now:



🔥 That's already much closer to a serious codebase RAG architecture.

---

15. But we still have a problem

Imagine our hybrid retriever returns:

```
1. login_user()
2. verify_password()
3. old_login_user()
4. test_login()
5. login_helper()
```

They're all somewhat relevant.

But maybe:

`old_login_user()`

is deprecated.

And:

`test_login()`

is a test, not production code.

And:

`login_helper()`

is less relevant than another result ranked #6.

So simply doing hybrid retrieval still isn't enough.

We need another stage:

Reranking

The pipeline becomes:

```
Vector + BM25
       ↓
Candidate retrieval
       ↓
Reranker
       ↓
Best candidates
```

And this is our **next lesson**.

🧠 One thing I want you to remember

Don't memorize BM25's formula yet.

Understand this:

> **Embeddings answer: "What code means something similar?"**
>
> **BM25 answers: "What code contains important words/identifiers from the query?"**
>
> **Hybrid retrieval combines both signals.**
>
> **Reranking then takes those candidates and asks: "Which ones are actually the best matches?"**

Next we'll learn **reranking**, including why a second model is often used, what a **cross-encoder** does, and why we don't just use the embedding model again.

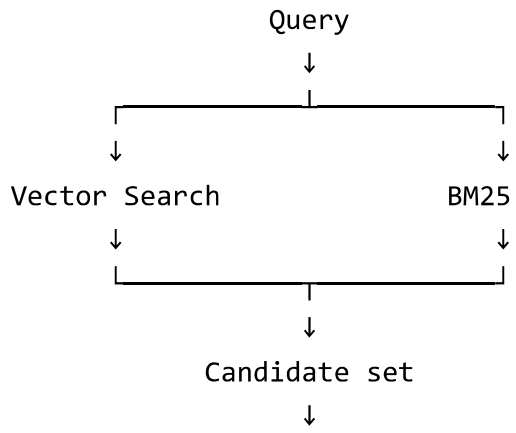
You said:

next

ChatGPT said:

Lesson 5 — Reranking: Getting the *Right* Results

We now have:



```
????
  ↓
 LLM
```

That `????` is **reranking**.

And this is an important distinction:

> **Retrieval finds candidates. Reranking decides which candidates are actually best.**

---

1. Why do we need reranking?

Suppose the developer asks:

> **"How does the application verify a user's password before creating a session?"**

Our hybrid retrieval gives us:

```
1. login_user()
2. verify_password()
3. create_session()
4. reset_password()
5. authenticate_request()
6. password_validator_test()
7. old_login_function()
```

All of these might have some relationship to:

```
password
authentication
session
```

But they're not equally useful.

The LLM doesn't necessarily need all seven.

We want something like:

```
1. login_user()
2. verify_password()
3. create_session()
```

So we need a better judge.

---

2. Retrieval and reranking solve different problems

Think about a Google search.

When you search:

> "best laptop for programming"

Google doesn't examine every webpage with an extremely expensive model.

It first retrieves a manageable set of candidates.

Then it can do more sophisticated ranking.

Same idea here.

Stage 1 — Retrieval

Fast:

```
1,000,000 chunks
       ↓
   retrieve
       ↓
   100 chunks
```

Stage 2 — Reranking

More expensive but more accurate:

```
100 candidates
       ↓
   reranker
       ↓
   10 best chunks
```

So:

```
Retrieval = recall
Reranking = precision
```

This is a very useful interview concept.

---

3. Why not use the embedding model again?

Good question.

Our embedding model does:

```
query → vector
chunk → vector
```

Then:

```
similarity(query_vector, chunk_vector)
```

The query and document are encoded **independently**.

For example:

```
Query:
"How is the password verified?"

        ↓

[0.21, -0.44, ...]
```

And:

```
Chunk:
verify_password()

        ↓

[0.18, -0.40, ...]
```

Then cosine similarity says:

```
0.89
```

That's fast.

But it doesn't deeply compare the query and chunk together.

4. Enter the Cross-Encoder

A **cross-encoder** takes the query and candidate **together**.

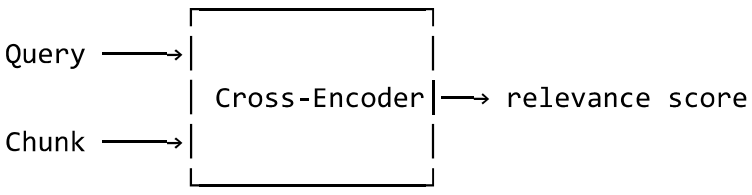
Instead of:

```
Query → vector

Chunk → vector

compare
```

we do:



For example:

```
Query:
"How is the password verified?"

Candidate:
def verify_password(password, password_hash):
    return bcrypt.checkpw(password, password_hash)
```

The model sees both together and predicts something like:

```
relevance = 0.96
```

Another:

```
Candidate:
def send_password_reset_email(...):
    ...
```

might get:

```
relevance = 0.18
```

5. Why can the cross-encoder be more accurate?

Because it can model **interactions between the query and the code**.

Consider:

Query

```
How is a password checked before login?
```

Candidate A

```
def login_user(username, password):
    user = find_user(username)

    if verify_password(password, user.password_hash):
        return create_session(user)
```

Candidate B

```
def reset_password(email):
    send_reset_email(email)
```

Both contain:

```
password
```

A simple keyword system might like both.

An embedding system may also find both semantically related.

But a cross-encoder can directly examine:

```
Query ↔ Candidate A
```

and:

```
Query ↔ Candidate B
```

and determine that A actually answers the question.

---

6. The tradeoff

Here's the catch.

Suppose we have:

```
1,000,000 chunks
```

If we run a cross-encoder against every chunk:

```
Query + Chunk 1 → model
Query + Chunk 2 → model
Query + Chunk 3 → model
...
Query + Chunk 1,000,000 → model
```

That would be painfully expensive.

So we **don't**.

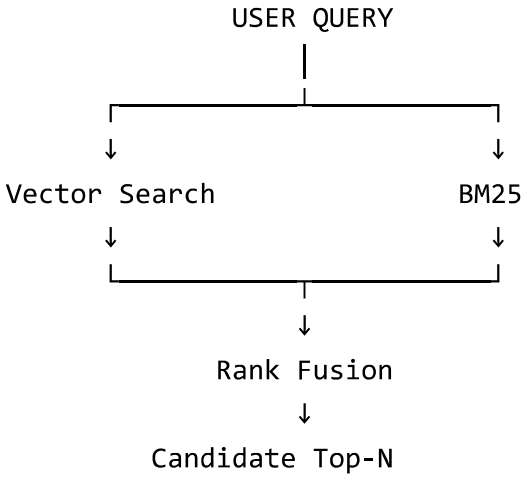
Instead:

```
1,000,000 chunks
       ↓
Vector + BM25
       ↓
100 candidates
       ↓
Cross-encoder
       ↓
Top 10
```

This is the key architecture.

---

7. So our retrieval pipeline now looks like this



```
                ↓
          Cross-Encoder
                ↓
          Reranked Top-K
                ↓
     Dependency Expansion
                ↓
       Context Assembly
                ↓
               LLM
```

Notice something important.

We have **three different jobs** now:

Vector search

```
Semantic recall
```

BM25

```
Exact-term recall
```

Reranker

```
Fine-grained relevance
```

That's a much better mental model than simply saying:

> "We use RAG."

---

8. Let's use a concrete example

Query:

> **"Where is the user's password checked before creating a session?"**

Suppose hybrid retrieval produces:

Candidate	Retrieval rank
`login_user()`	1
`verify_password()`	2
`create_session()`	3
`reset_password()`	4
`authenticate_request()`	5

Now reranker evaluates:

```
Query + login_user()
Query + verify_password()
Query + create_session()
Query + reset_password()
Query + authenticate_request()
```

It might produce:

Candidate	Reranker score
`verify_password()`	0.97
`login_user()`	0.94
`create_session()`	0.71
`authenticate_request()`	0.42
`reset_password()`	0.12

Now our final ranking becomes:

1. `verify_password()`
2. `login_user()`
3. `create_session()`

Much better.

---

9. Important: reranking doesn't necessarily mean "another LLM"

This is a common misconception.

A reranker can be a relatively small specialized model.

For example, there are models specifically trained for:

`(query, document) → relevance`

They aren't necessarily generating text.

Their job is simply:

> **Score how relevant this candidate is to this query.**

That's why they can be much cheaper than asking a generative LLM to judge every candidate.

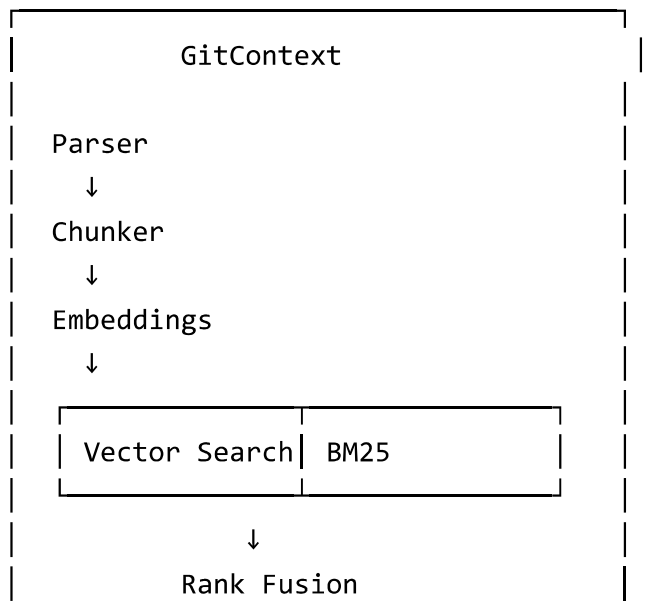
---

10. The architecture is becoming modular

This is something I want you to notice.

We aren't building one giant "AI model."

We're building a system:



```
│              ↓                        │
│           Reranker                    │
│              ↓                        │
│       Dependency Expansion            │
│              ↓                        │
│       Context Assembly                │
│              ↓                        │
│             LLM                       │
└───────────────────────────────────────┘
```

This is **AI engineering** rather than simply "calling an AI API."

---

11. There's another problem hiding here

Suppose reranking gives us:

```
verify_password() → 0.97
login_user() → 0.94
create_session() → 0.71
```

Great.

But what if:

`verify_password()`

is from an **old deprecated implementation**?

And:

`auth_v2.py`

contains the current implementation.

Our reranker might still rank the old one highly because it is semantically relevant.

So relevance isn't the only consideration.

We may need:

```
semantic relevance
+
exact match
+
code freshness
```

```
+
file importance
+
dependency relationships
+
language
+
repository branch
```

This leads us toward **metadata-aware retrieval and ranking**.

---

12. And there's an even bigger question

Eventually GitContext will answer:

> "How does authentication work?"

But how do we know whether the answer is **actually correct**?

We can't just look at it and say:

> "Looks good."

We need measurements.

For example:

```
Did we retrieve the correct function?

Did Top-K contain the relevant chunk?

Did the final answer use the correct evidence?

Did the answer hallucinate?

Did it cite the correct file?
```

This leads us to one of the most important parts of the project:

Evaluation

And this is where your GitContext project starts becoming much more serious than a typical RAG demo.

---

13. Our complete pipeline so far



We're now ready for the next major topic.

Next: Lesson 6 — RAG Evaluation

We'll build a tiny evaluation dataset such as:

```
Question:
"How is a user's password verified?"

Expected chunk:
auth.py → verify_password()

Retrieved:
auth.py → login_user()
auth.py → verify_password()
```

Then we'll learn:

- **Recall@K**
- **Precision@K**
- MRR
- answer correctness
- citation correctness
- hallucination
- retrieval failure vs generation failure
- how to systematically discover that our RAG system is bad

This is extremely important because an interviewer can ask:

> **"How do you know your RAG system actually works?"**

And "I tested it manually" is not a strong engineering answer.

Continued from RAG PROJECT